

SĂPTĂMÂNA 5

Simularea cu evenimente discrete

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Unde am rămas

- ✓ Cozi: sosiri Poisson, **ρ = λ/(cμ)**, legea lui Little **L = λW** (S4)
- ✓ Formule exacte doar pentru M/M/1, M/M/c, M/M/1/K, M/G/1 – și doar în **regim staționar**
- ✓ Variabilitatea costă timp; rețelele adună timpii; percentilele contează
- ✓ Întrebarea de final: cum simulăm o zi întreagă la bancă în câteva **milisecunde**?

La finalul acestei săptămâni veți putea…

1. să explicați **starea**, **evenimentul**, **ceasul simulat** și **lista evenimentelor viitoare**
2. să comparați abordarea **time-stepping** cu abordarea **event-driven**
3. să urmăriți „de mână” o simulare și să calculați **medii ponderate în timp**
4. să scrieți un **motor DES** în ~40 de linii de Python și să-l validați pe M/M/1
5. să descrieți cele trei abordări: evenimente, activități, **procese**
6. să modelați sisteme cu resurse în **SimPy**: intersecție, producție, urgențe

Ora 1

Stare, eveniment, ceas

1. Ce este simularea cu evenimente discrete (DES)
2. Time-stepping vs event-driven: bila în cutie
3. Anatomia unui simulator: ceas, listă de evenimente, acțiuni
4. Graful activităților (Holban) și o urmă făcută de mână

De ce nu ne ajung formulele din S4

- Sosiri care **nu** sunt Poisson: vârfuri la prânz, programări, rafale de cereri
- Serviri cu distribuții oarecare, **priorități**, clienți care renunță și pleacă din coadă
- Resurse care se **defectează**, ture de lucru, ghișee care se deschid și se închid
- Rețele cu **blocare**: o stație plină oprește stația din amonte
- Ne interesează **percentilele** și regimul tranzitoriu (dimineața, după un vârf), nu doar media

Ideea

Construim un **program** care imită sistemul pas cu pas și **măsurăm** în program exact mărimile care ne interesează: W, L, ρ, p95.

Simularea cu evenimente discrete

Discrete-event simulation (DES)

Starea sistemului se schimbă **doar în anumite momente**, numite **evenimente**. Între două evenimente consecutive starea rămâne constantă, așa că simulatorul **sare** direct de la un eveniment la următorul.

Cernazean L7; Cambridge CSM; Law, 2015

- Timpul simulat este **continuu** (orice număr real), dar evenimentele sunt **discrete** (numărabile)
- Exemple de evenimente: sosește un client, se termină o servire, un semafor trece pe verde, se defectează o mașină
- Costul simulării depinde de **numărul de evenimente**, nu de lungimea intervalului simulat

Vocabularul simulării

1 Entitate

Obiectul care circulă prin sistem: client, pachet, piesă, pacient.

2 Atribut

O proprietate a entității: momentul sosirii, prioritatea, tipul cererii.

3 Stare

Tot ce trebuie să știm acum ca să continuăm: câți clienți, ce servere sunt ocupate.

4 Eveniment

Moment în care starea se schimbă: sosire, terminarea servirii.

5 Activitate

Un interval de durată cunoscută la început: o servire, o călătorie.

6 Proces

Succesiunea de evenimente și activități a unei entități, de la sosire la plecare.

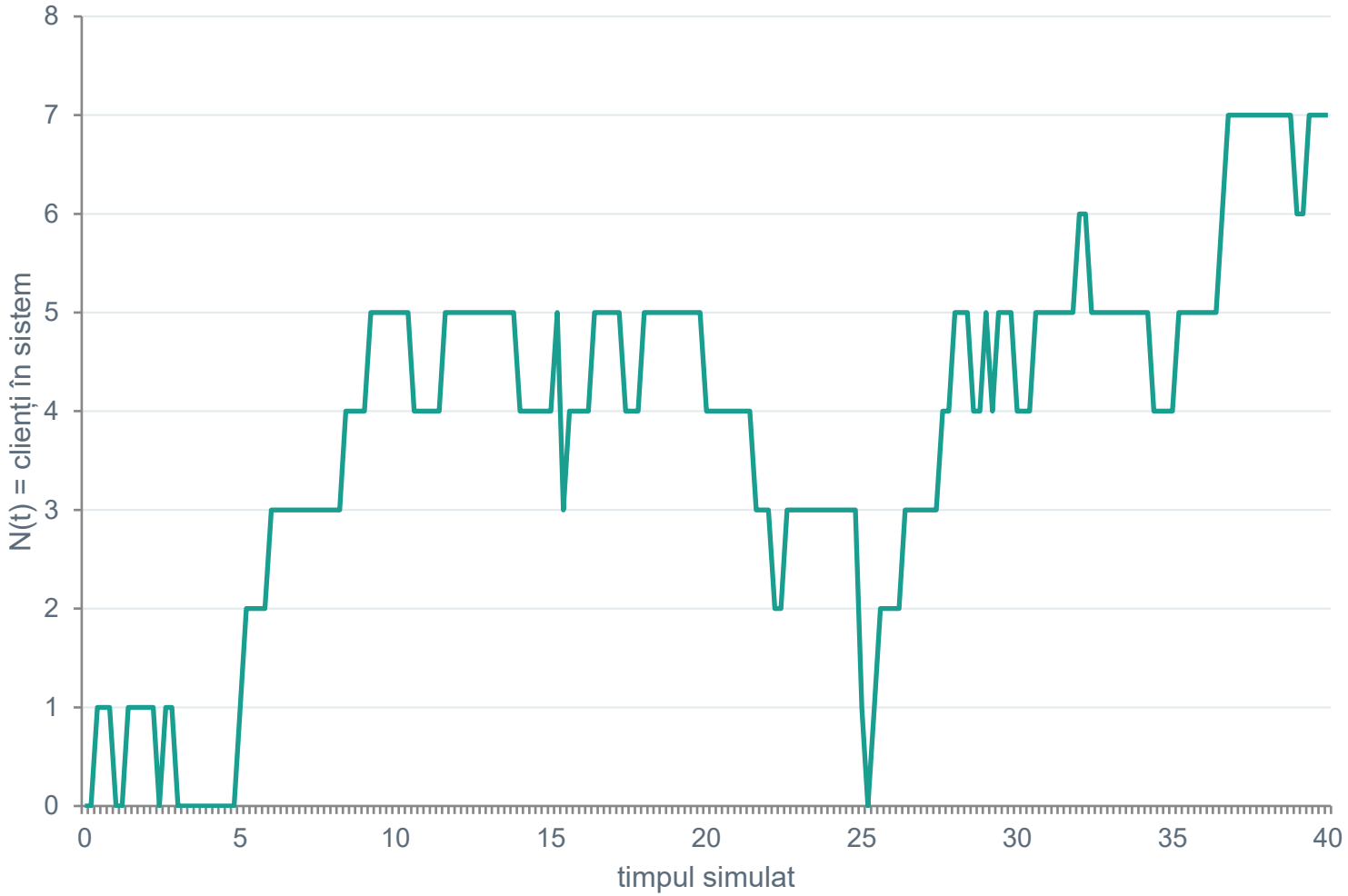
7 Resursă

Ce se folosește și se eliberează: casier, CPU, medic, loc de parcare.

8 Coadă

Entitățile care așteaptă o resursă, ordonate după o disciplină.

Starea se schimbă doar la evenimente (M/M/1, ρ = 0,8)



- N(t) este o funcție **în trepte**
- Fiecare treaptă în sus = o **sosire**; fiecare treaptă în jos = o **plecare**
- Între trepte nu se întâmplă nimic: simulatorul **sare** peste aceste intervale
- Primele 40 de unități de timp: ~60 de evenimente

O zi la bancă: câți pași de calcul?

≈ 600

evenimente: 300 de clienți × (sosire + plecare) – atât are de procesat un simulator DES

28 800

pași de câte 1 secundă pentru 8 ore – dacă avansăm timpul cu un pas fix

2,88 mil.

pași de câte 10 ms, dacă vrem să nu ratăm evenimente apropiate

- Cu pas fix, aproape toți pașii nu schimbă nimic: doar verificăm că „nu s-a întâmplat nimic”
- Cu evenimente, fiecare pas de calcul corespunde unei schimbări reale de stare

Două moduri de a avansa timpul

Time-stepping (pas fix)

- $t \leftarrow t + \Delta t$, apoi actualizăm **toată** starea
- simplu de scris; natural pentru fenomene **continue** (S7)
- Δt mare: ratăm sau întârziem evenimente
- Δt mic: foarte mulți pași inutili

vs

Event-driven (evenimente)

- $t \leftarrow$ momentul **următorului eveniment**
- natural pentru cozi, rețele, producție, logistică
- **exact** în momentele evenimentelor
- cere să știm **când** va avea loc următorul eveniment

Atenție la termeni (Cernazean): într-o simulare DES timpul este **continuu**; într-o simulare a unui model continuu (EDO), timpul este de fapt **discretizat** cu pasul Δt.

Exemplul lui Cernazean

O bilă într-o cutie

1. Mișcare fără forțe, ciocniri perfect elastice
2. Abordarea „forță brută”: pas fix Δt
3. Abordarea inteligentă: calculăm momentul următoarei ciocniri
4. Comparație: număr de pași și precizie

Fizica simplă: mișcarea liberă

$$x(t) = v_x \cdot t + x(0), \quad y(t) = v_y \cdot t + y(0)$$

cutia	pătrat de **16 m × 16 m**; ciocniri perfect elastice: la perete se inversează componenta vitezei
start	poziția (4, 5) la t = 0, viteza **v = (2; 0,9)** m/s
întrebarea	unde este bila după **3 minute** (t = 180 s)?

Fără pereți, răspunsul ar fi imediat: (364; 167). Pereții schimbă viteza și fac problema interesantă: câte ciocniri au loc și **când**?

Abordarea „forță brută”: pas fix Δt

```
def time_step(dt, T=180, L=16):
    x, y, vx, vy = 4.0, 5.0, 2.0, 0.9
    for _ in range(int(round(T / dt))):
        if x <= 0 or x >= L:   # perete vertical?
            vx = -vx
        if y <= 0 or y >= L:   # perete orizontal?
            vy = -vy
        x += vx * dt
        y += vy * dt
    return x, y

print(time_step(0.01))   # 18 000 de pași
```

- La fiecare pas: **verificăm** pereții, apoi **mutăm** bila
- Ciocnirea este detectată abia **după** ce bila a trecut de perete
- Eroarea depinde de Δt: fiecare ciocnire adaugă o mică întârziere
- Pentru t = 180 s și Δt = 0,01: **18 000** de iterații

Abordarea inteligentă: când are loc următoarea ciocnire?

$$t_n = (L - y)/v_y = 12{,}22 \qquad t_e = (L - x)/v_x = 6{,}00$$

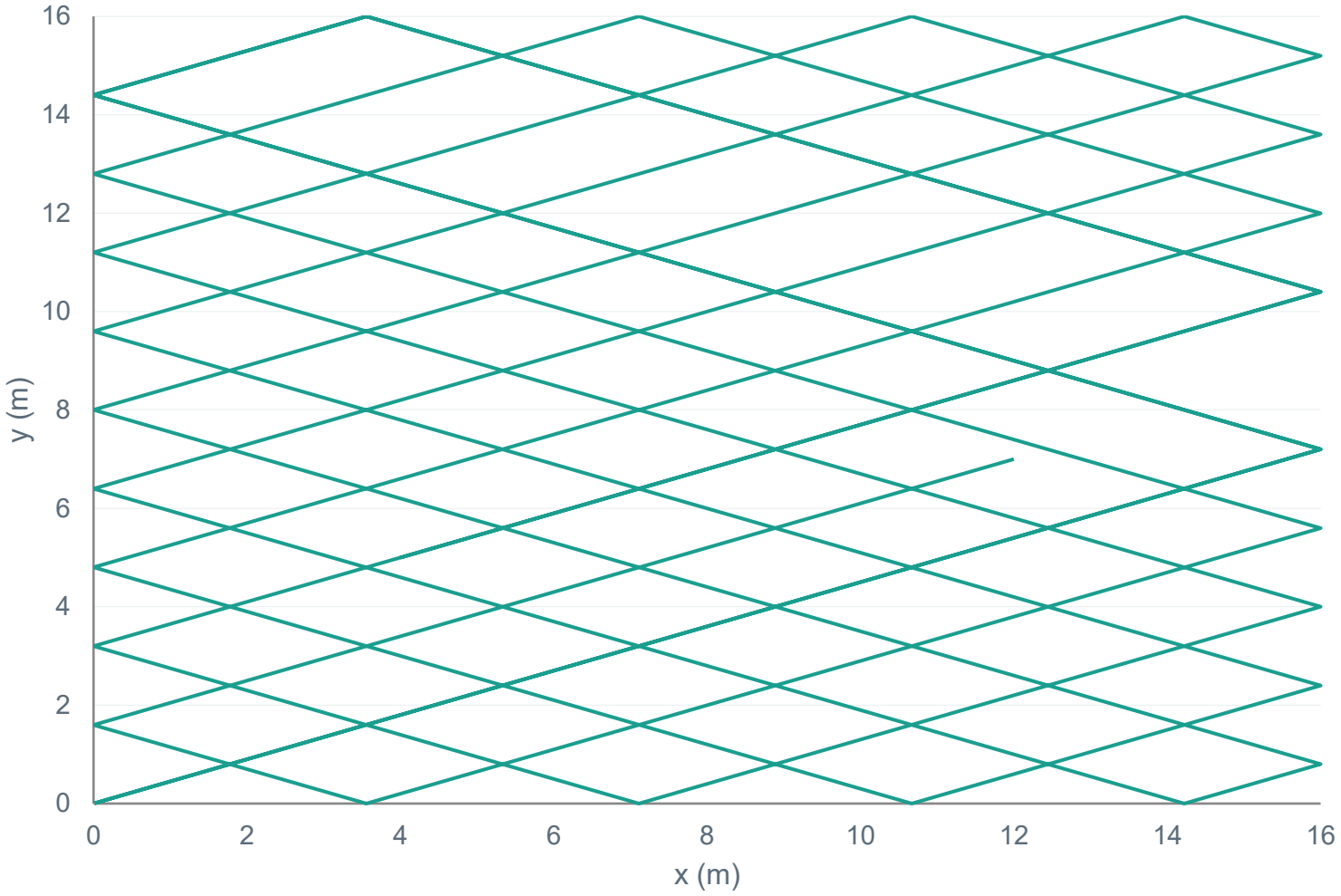
nord, est momentele în care bila ar atinge peretele de sus (12,22 s) și pe cel din dreapta (6,00 s)

sud, vest $t = -y/v_y = -5{,}56$ și $t = -x/v_x = -2{,}00$: **negative**, adică în trecut – le ignorăm

următorul minimul pozitiv: **6,00 s**, peretele din dreapta → sărim direct acolo și inversăm v_x

Între ciocniri mișcarea este cunoscută **exact**, deci nu avem nimic de simulat pas cu pas: calculăm momentul următorului eveniment și sărim la el.

Traiectoria pe 3 minute: 32 de ciocniri



- **32** de evenimente (22 cu pereții verticali, 10 cu cei orizontali)
- Poziția finală: **(12; 7)** – exact soluția analitică
- La t = 30 s bila lovește **colțul** (0, 0) și se întoarce pe același drum
- Fiecare segment = o singură iterație a simulatorului

Comparație: pas fix vs evenimente (t = 180 s)

Metoda	Iterații	Poziția finală	Eroarea (m)
Pas fix, Δt = 0,1	1 800	(7,60; 5,90)	4,535
Pas fix, Δt = 0,01	18 000	(11,56; 6,89)	0,454
Pas fix, Δt = 0,001	180 000	(11,96; 6,99)	0,045
Evenimente	**32**	(12; 7)	**0 (exact)**

- Cu pas fix, eroarea scade doar **proporțional** cu Δt: de 10 ori mai precis costă de 10 ori mai mulți pași
- Varianta cu evenimente este **exactă** și de aproximativ **5 625** de ori mai scurtă decât pasul de 0,001 s
- Rezultat similar cu tabelul lui Cernazean: 18 000 de iterații cu eroare de câteva procente vs 34 de pași exacți

Când folosim fiecare abordare

- **Evenimente**: între evenimente starea e constantă sau se calculează **exact** (cozi, rețele, logistică, protocoale)
- **Pas fix**: forțe care acționează continuu (gravitație, arcuri, reacții chimice) – ecuații diferențiale, **S7**
- **Hibrid**: pas fix pentru mișcare + evenimente pentru ciocniri, schimbări de regim, comenzi
- Prima simulare de dinamică moleculară (Alder & Wainwright, **1957**) a fost condusă de evenimente: sfere dure care se ciocnesc
- Jocurile video folosesc de obicei pas fix (cadre), cu detecție de coliziuni între cadre

? **Regula practică**

Dacă poți calcula **când** se schimbă starea, folosește evenimente. Dacă starea se schimbă **tot timpul**, folosește pas fix.

Verificare

? Pentru care dintre sistemele următoare este cea mai potrivită o simulare **condusă de evenimente**?

A mișcarea unei planete în jurul Soarelui

B ghișeele unei bănci într-o zi de lucru

C încălzirea unei camere cu un calorifer

D curgerea apei printr-o conductă

Anatomia unui simulator

Ceas, evenimente, acțiuni

1. Evenimente și acțiuni; endogen vs exogen
2. Ceasul simulat și lista evenimentelor viitoare
3. Algoritmul DES și condiția de oprire
4. Graful evenimentelor pentru o coadă

Evenimentul și acțiunea lui

Eveniment = (moment, tip, date)

Fiecărui eveniment îi corespunde o **acțiune**: o funcție care modifică starea S și **lista evenimentelor viitoare** Q, adică a: (S, Q) → (S, Q). Evenimentul este **cauza**, acțiunea este **efectul**.

Cernazean L7

- O acțiune poate **adăuga** evenimente noi (o sosire programează sosirea următoare)
- O acțiune poate **anula** evenimente (un client renunță, deci plecarea lui nu mai are loc)
- Toată „logica” modelului se află în aceste acțiuni – restul simulatorului este mereu același

Evenimente endogene și exogene

Endogene (din interior)

- consecințe ale evoluției sistemului
- ex.: se termină o servire, un ghișeu devine liber
- sunt programate de **acțiunile** altor evenimente
- nu pot fi cunoscute dinainte

Exogene (din exterior)

- vin din afara sistemului – ca **condițiile la limită**
- ex.: sosirea clienților, sosirea mașinilor la semafor
- nu depind de starea sistemului
- pot fi **generate dinainte**, din date reale sau pseudo-aleator (S2)

Pentru că evenimentele exogene nu depind de stare, putem folosi **aceeași** listă de sosiri pentru mai multe variante ale sistemului – o comparație corectă (S6).

Ceasul simulat și lista evenimentelor viitoare

Ceasul și FEL

Ceasul simulat t indică momentul curent din lumea modelului; nu are legătură cu timpul real de execuție. **Lista evenimentelor viitoare** (FEL, future event list) conține toate evenimentele programate, ordonate după moment – o **coadă de priorități**.

- Ceasul **sare** direct la momentul primului eveniment din FEL; nu avansează niciodată înapoi
- Operațiile de bază: **inserare** și **extragerea minimului** – de milioane de ori într-o simulare
- Structura de date potrivită (heap) este esențială pentru viteză – ora 2

Algoritmul DES

Aceeași buclă pentru orice model (Cambridge CSM; Cernazean L7):



Esențial: se extrage **întotdeauna** evenimentul cu cel mai mic moment. Altfel un eveniment din „viitor” ar putea modifica starea înaintea unuia din „trecut” – o încălcare a **cauzalității**.

Algoritmul în pseudocod (Cernazean L7)

```
# inițializare
t = t0
Q = evenimente_exogene()   # ex.: sosirile
S = stare_initiala(t0)

# evoluție
while not conditie_oprire(t, S, Q):
    e = Q.extrage_minim()   # următorul eveniment
    t = e.moment            # sărim la el
    S, Q = e.actiune(S, Q)  # starea și lista

# rezultate: statisticile colectate în acțiuni
```

- Bucla are **patru linii** – tot restul este logica modelului
- Q = lista evenimentelor viitoare (coadă de priorități)
- Acțiunea primește și întoarce **starea** și **lista**
- În ora 2 transformăm acest pseudocod în Python real

Condiția de oprire

- Un **orizont de timp**: t ≥ 8 ore (o zi de lucru), t ≥ 1 an
- Un **număr de entități**: după 10 000 de clienți serviți
- Lista **goală**: nu mai există evenimente (toți clienții au plecat, bila… nu se oprește niciodată)
- Un **eveniment special** de sfârșit, pus în FEL la început: „închiderea băncii la 17:00”
- O **precizie statistică**: oprim când intervalul de încredere e suficient de îngust (S6)

C **Cambridge**

După ora T nu mai primim clienți noi, dar îi servim pe cei rămași – până se golește sistemul. Oprirea trebuie să reflecte **realitatea** modelată.

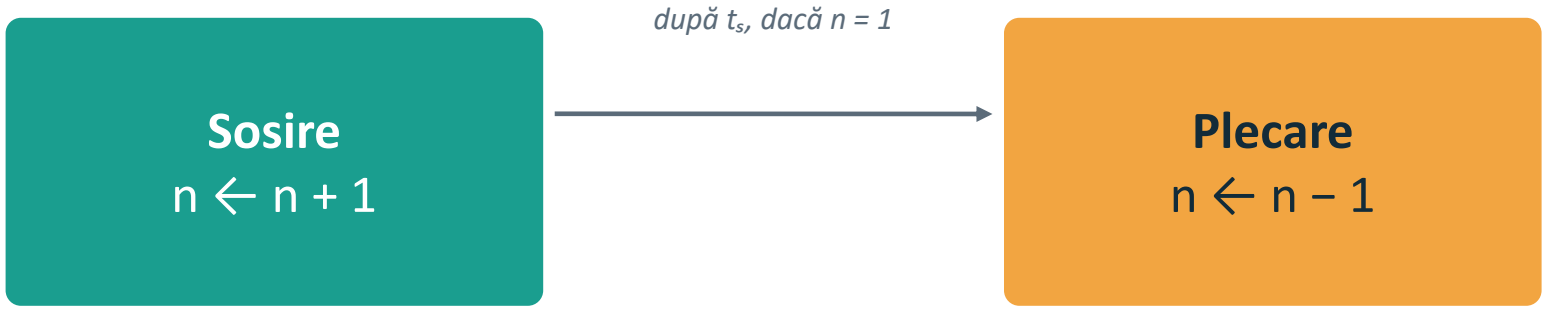
Ce variabile ține un simulator (Cambridge)

- **Timpul**: ceasul simulat t
- **Contoare**: câte sosiri și câte plecări au avut loc până la momentul t
- **Starea sistemului**: n = clienți în sistem, ce servere sunt ocupate, cine e în coadă
- **Acumulatori statistici**: suma timpilor de așteptare, aria de sub N(t), timpul cât serverul a fost ocupat
- La final: **mediile** se calculează din acumulatori, fără să păstrăm toată istoria

✓ **Verificare utilă**

Conservarea: **sosiri = plecări + clienți încă în sistem**. Dacă nu se verifică, simulatorul are o eroare.

Graful evenimentelor pentru o coadă cu un server



Sosire → Sosire: după t_a (timpul până la următoarea sosire), **întotdeauna**

Plecare → Plecare: după t_s, **dacă n > 0** (începe servirea următorului client)

Cum se citește

Nod = tip de eveniment și schimbarea de stare pe care o produce.

Arc = „programează”: după întârzierea scrisă pe arc, dacă e adevărată condiția.

Schruben (1983): orice model DES poate fi desenat astfel.

Logica evenimentelor pentru o coadă cu un server

Eveniment	Schimbă starea	Programează
Sosire	$n \leftarrow n + 1$; dacă serverul e ocupat, clientul intră în **coadă**	următoarea sosire la $t + t_a$; dacă serverul era liber: **plecarea** la $t + t_s$
Plecare	$n \leftarrow n - 1$; serverul se eliberează	dacă coada nu e goală: primul din coadă începe servirea → **plecarea** lui la $t + t_s$

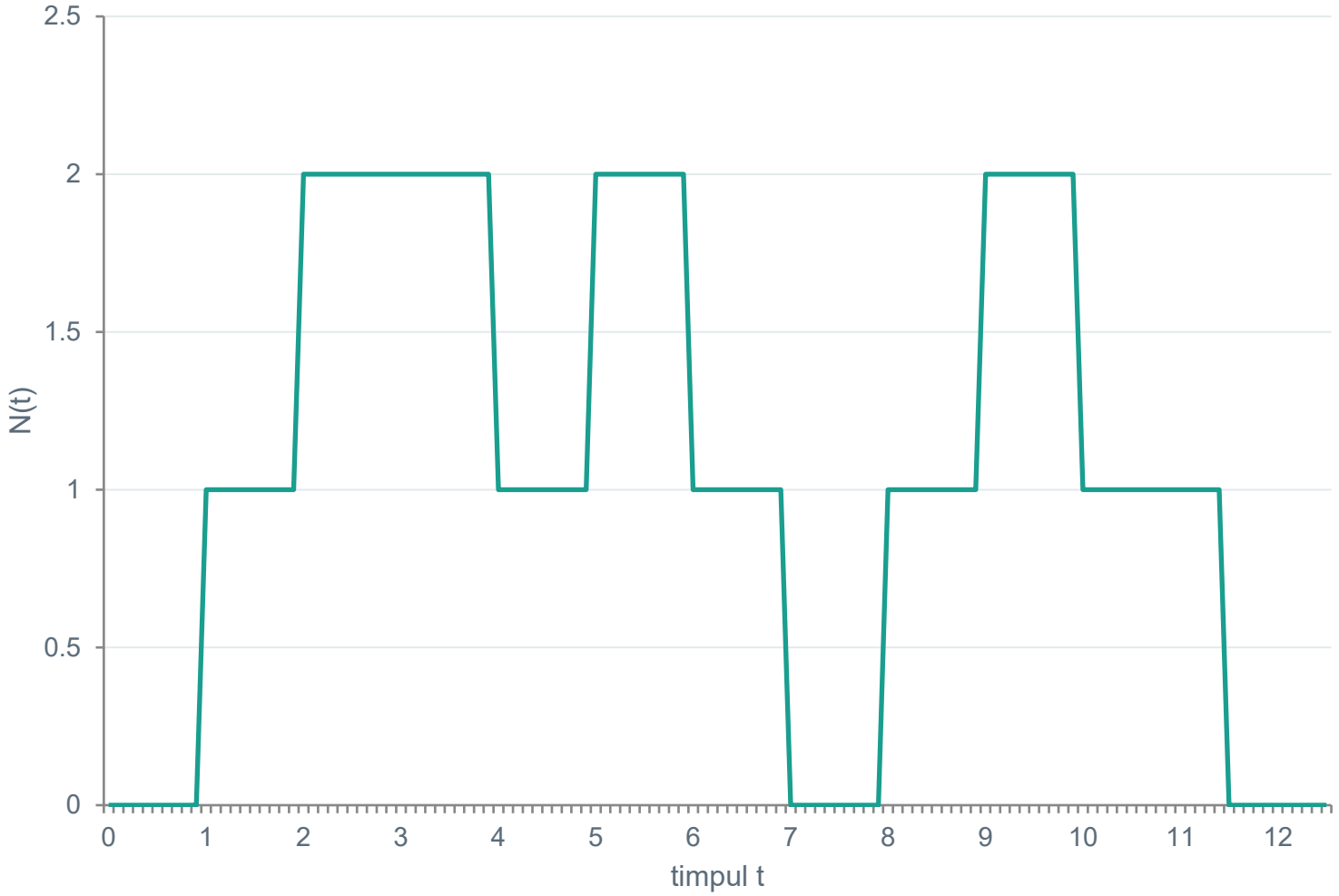
- Doar **două** tipuri de evenimente ajung pentru orice coadă cu un server, oricare ar fi distribuțiile t_a și t_s
- Pentru M/M/1, $t_a \sim \text{Exp}(\lambda)$ și $t_s \sim \text{Exp}(\mu)$; pentru alte modele schimbăm doar **generatoarele**, nu logica
- Statisticile se actualizează la fiecare eveniment: aria de sub N(t), timpul ocupat, așteptarea fiecărui client

O urmă făcută de mână: 5 clienți la un ghișeu

t	Eveniment	N	Server	Coadă	FEL după eveniment
1	S1	1	ocupat	–	S2(2), P1(4), S3(5), S4(8), S5(9)
2	S2	2	ocupat	C2	P1(4), S3(5), S4(8), S5(9)
4	P1	1	ocupat	–	S3(5), P2(6), S4(8), S5(9)
5	S3	2	ocupat	C3	P2(6), S4(8), S5(9)
6	P2	1	ocupat	–	P3(7), S4(8), S5(9)
7	P3	0	liber	–	S4(8), S5(9)
8	S4	1	ocupat	–	S5(9), P4(10)
9	S5	2	ocupat	C5	P4(10)
10	P4	1	ocupat	–	P5(11,5)
11,5	P5	0	liber	–	–

Sosiri: 1, 2, 5, 8, 9; serviri: 3; 2; 1; 2; 1,5. S = sosire, P = plecare (S2 = sosirea clientului 2).

N(t) pentru urma de mână: aria de sub curbă = 13,5



- Aria = Σ N·(durata intervalului) = **13,5** clienți × u.t.
- Serverul ocupat: [1, 7] și [8; 11,5] → **9,5** u.t.
- Sistemul gol între 7 și 8: serverul stă liber
- Durata totală: **11,5** u.t.

Două feluri de medii

$$L = (1/T)\cdot\int N(t)\,dt \qquad W = (1/n)\cdot\Sigma\, W_i$$

în timp media **ponderată în timp**: L = 13,5 / 11,5 = **1,17** clienți; ρ = 9,5 / 11,5 = **0,83**

pe clienți media peste **entități**: W = (3 + 4 + 2 + 2 + 2,5)/5 = **2,7**; Wq = **0,8**

Little λ = 5/11,5 = 0,435; λ·W = 1,17 = L ✓

Greșeală frecventă: media valorilor lui N **la evenimente** (1, 2, 1, 2, 1, 0, …) nu este L – intervalele au durate diferite.

Rezultatele pe clienți

Client	Sosire	Servire	Începe	Pleacă	Așteaptă	În sistem
C1	1	3	1	4	0	3
C2	2	2	4	6	2	4
C3	5	1	6	7	1	2
C4	8	2	8	10	0	2
C5	9	1,5	10	11,5	1	2,5

- Așteptarea = începutul servirii − sosirea; timpul în sistem = plecarea − sosirea
- Doi clienți din cinci nu au așteptat deloc; media așteptării: **0,8** u.t.
- Aceleași rezultate le dă și recurența lui Lindley din S4 – pentru o coadă FIFO cu un server

Alt exemplu: spălătoria auto cu două boxe (Cambridge)

t	Eveniment	Acțiune	Coadă după
6	sosește mașina 1	mașina 1 intră în CW1	–
11	sosește mașina 2	mașina 2 intră în CW2	–
12	sosește mașina 3	mașina 3 așteaptă	mașina 3
14	CW1 termină mașina 1	mașina 3 intră în CW1	–
17	sosește mașina 4	mașina 4 așteaptă	mașina 4
19	sosește mașina 5	mașina 5 așteaptă	mașina 4, mașina 5
21	CW2 termină mașina 2	mașina 4 intră în CW2	mașina 5
22	CW1 termină mașina 3	mașina 5 intră în CW1	–
30	CW1 termină mașina 5	CW1 rămâne liberă	–
31	CW2 termină mașina 4	CW2 rămâne liberă	–

Boxa CW1 spală o mașină în 8 min, CW2 în 10 min; mașinile sosesc la 6, 11, 12, 17 și 19 min.

Verificare

? Ceasul arată **t = 6**. Lista evenimentelor viitoare conține: S4(8), P3(7,5), S5(9). Ce face simulatorul la pasul următor?

A avansează ceasul la t = 6,1

B execută S4 și mută ceasul la t = 8

C execută P3 și mută ceasul la t = 7,5

D execută toate cele trei evenimente

Holban, cap. 2

Graful activităților

1. Blocurile grafului: sosire, așteptare, servire, plecare, resurse
2. Studiu de caz: benzinăria cu 2 pompe
3. Varianta cu magazin: rutare probabilistică
4. Oficiul poștal (Cernazean): stare, evenimente, optimizare

Blocurile grafului activităților (Holban)

1 Sosire

Sursa clienților: cum sosesc (timpul mediu, distribuția) și ce atribute primește fiecare client.

2 Așteptare

Șirul format în fața unei resurse; blocul specifică disciplina (FIFO, priorități).

3 Servire

O resursă cu una sau mai multe stații; specifică durata ocupării (medie, distribuție).

4 Plecare

Ieșirea clienților din sistem; aici se colectează de obicei timpul total petrecut.

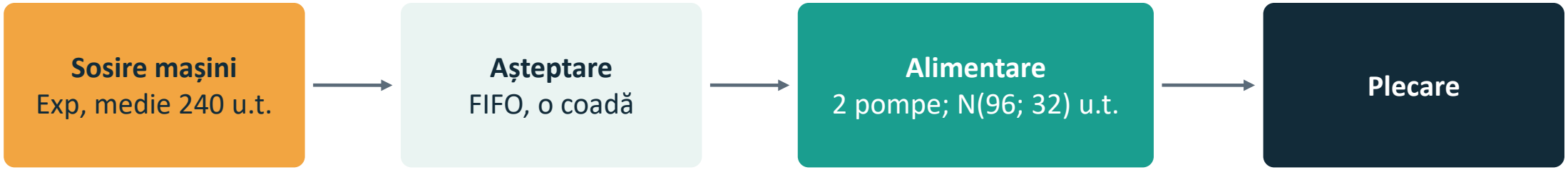
5 Alocare / eliberare

Cere sau returnează o cantitate dintr-un depozit de resurse (memorie, locuri de parcare).

6 Arce

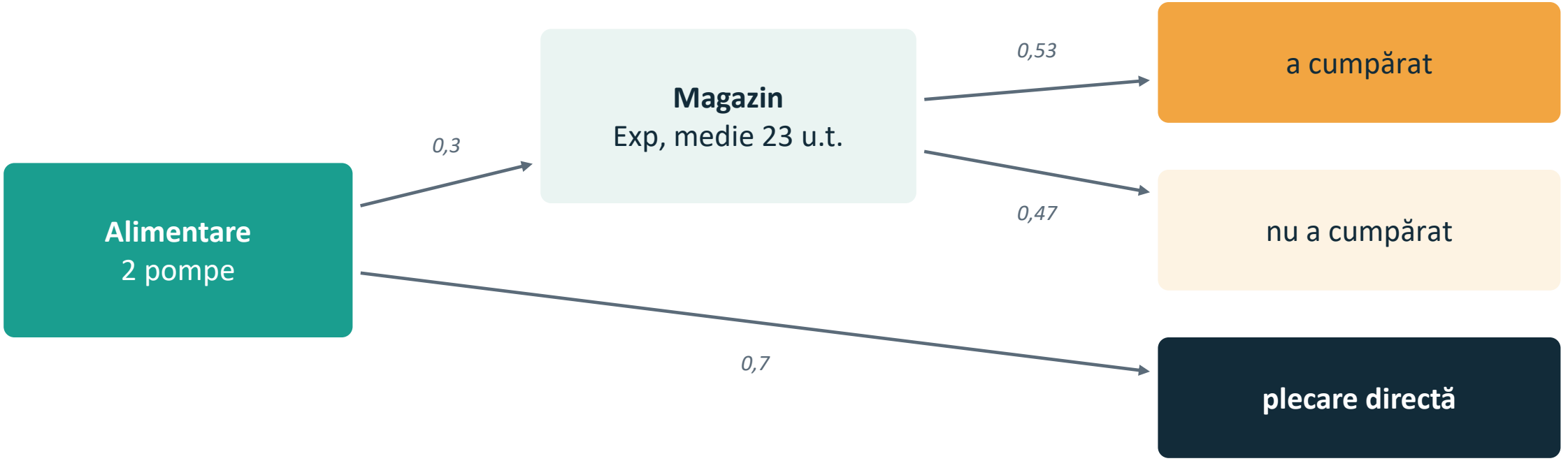
Succesiunea activităților; pot purta probabilități de rutare (0,4 / 0,6).

Studiu de caz: benzinăria cu două pompe (Holban)



Ce aflăm prin simulare: timpul mediu de așteptare, lungimea medie și maximă a cozii, gradul de încărcare al pompelor, capacitatea de trecere (mașini pe unitatea de timp), timpul mediu de răspuns.
Variante (Holban): pompe individuale (ce pompă se ocupă prima?), două cozi separate (40% / 60%), două clase de clienți (autoturisme și autobuze, cu timpi diferiți).

Varianta 4: magazinul de lângă benzinărie



Staționarea în parcarea magazinului este o **așteptare cu timp asociat**: mașina ocupă un loc 23 u.t. în medie. Simularea dă numărul mediu și maxim de mașini parcate – adică **câte locuri** trebuie să aibă parcarea.

Oficiul poștal (Cernazean): un model pentru optimizare

Modelul

- n ghișee, fiecare **închis / deschis / ocupat**, după un program zilnic
- clienți cu momentul sosirii și durata cererii p_j
- stare: vectorul ghișeelor + coada FIFO
- evenimente: Deschide, Procesează, Închide (endogene); Sosire (exogen)

Întrebările

- ce program al ghișeelor **minimizează** coada?
- când deschidem un ghișeu în plus, înainte ca coada să depășească un prag?
- merită o coadă **separată** pentru cererile scurte?
- fiecare variantă = o simulare; sosiri aleatoare → mai multe rulări (**Monte Carlo**)

Simularea devine o **funcție obiectiv**: primește parametrii (programul ghișeelor) și întoarce performanța (lungimea cozii, așteptarea) – iar un algoritm de optimizare o poate apela de mii de ori.

Verificare

? În modelul oficiului poștal, care dintre evenimente este **exogen**?

A sosirea unui client

B terminarea procesării unei cereri

C deschiderea unui ghișeu după program

D trecerea unui ghișeu din „ocupat” în „deschis”

Verificare

? Într-o simulare de 10 minute, N(t) = 0 timp de 2 min, N(t) = 1 timp de 3 min și N(t) = 2 timp de 5 min. Care este numărul mediu de clienți L?

A 1

B 1,3

C 1,5

D 3

Rezumatul orei 1

- ✓ **DES**: starea se schimbă doar la evenimente; timpul simulat e continuu, simulatorul **sare** între evenimente
- ✓ Pas fix vs evenimente: bila în cutie – **32 de evenimente exacte** vs 18 000 de pași aproximativi
- ✓ Anatomie: **ceas**, **listă de evenimente viitoare** (coadă de priorități), acțiuni care schimbă starea și lista
- ✓ Evenimente endogene / exogene; graful evenimentelor; graful activităților (Holban)
- ✓ Statistici: **medii ponderate în timp** (L, ρ) și medii pe clienți (W); Little ca verificare

Ora 2

Un motor DES de la zero

1. Coada de priorități (heap) și bucla principală
2. Ghișeul M/M/1 simulat și comparat cu formulele din S4
3. Extensii: mai multe servere, priorități, clienți care renunță
4. Trei abordări: evenimente, activități, procese

Unde am rămas

- ✓ DES: starea se schimbă doar la **evenimente**; ceasul **sare** de la un eveniment la altul
- ✓ Componente: **ceas**, **listă de evenimente viitoare** (FEL), **acțiuni** care schimbă starea și lista
- ✓ Coada cu un server: doar două evenimente – **Sosire** și **Plecare**
- ✓ Statistici: medii **ponderate în timp** (L, ρ) și medii **pe clienți** (W)

Planul motorului

- Lista evenimentelor viitoare = un **heap** (modulul standard heapq)
- Un eveniment = tuplul **(moment, număr de ordine, acțiune, argumente)**
- Acțiunile sunt **funcții Python** obișnuite; ele pot programa alte evenimente
- Motorul (clasa Sim) nu știe nimic despre cozi: doar ține ceasul și lista
- Modelul (clasa MM1) ține starea și statisticile și definește acțiunile

M

Separarea responsabilităților

Același motor de ~10 linii rulează orice model: o coadă, o rețea, o intersecție. Schimbăm doar **modelul**.

Coada de priorități: heap-ul binar

Heap (min-heap)

Un arbore binar complet, păstrat într-o **listă**, în care fiecare nod este **mai mic sau egal** decât copiii lui. Minimul este mereu pe prima poziție.

- Inserare (heappush) și extragerea minimului (heappop): **O(log n)** pași
- Pentru 1 milion de evenimente în listă: ~20 de comparații pe operație, nu 1 milion
- Tuplurile se compară pe rând: întâi momentul, apoi numărul de ordine

heapq în cinci linii

```
import heapq

fel = []
heapq.heappush(fel, (8.0, "sosire C4"))
heapq.heappush(fel, (7.5, "plecare C3"))
heapq.heappush(fel, (9.0, "sosire C5"))

while fel:
    t, ev = heapq.heappop(fel)
    print(t, ev)

# 7.5 plecare C3
# 8.0 sosire C4
# 9.0 sosire C5
```

- Inserăm în **orice ordine**
- Extragem mereu **cel mai mic** moment
- Exact întrebarea de verificare din ora 1
- Nu sortăm lista: heap-ul ține doar ordinea minimă necesară

Motorul: clasa Sim (10 linii)

```
import heapq, itertools

class Sim:
    def __init__(self):
        self.now = 0.0
        self.fel = []                  # lista evenimentelor viitoare
        self.seq = itertools.count()   # 0, 1, 2, ...

    def schedule(self, delay, action, *args):
        heapq.heappush(self.fel,
            (self.now + delay, next(self.seq), action, args))

    def run(self, until=float("inf")):
        while self.fel and self.fel[0][0] <= until:
            self.now, _, action, args = heapq.heappop(self.fel)
            action(*args)              # executăm evenimentul
```

- schedule: programează o acțiune după o **întârziere**
- run: bucla principală – exact algoritmul DES din ora 1
- Oprire: lista goală sau orizontul until depășit
- Ceasul now avansează **doar** în run

De ce avem nevoie de numărul de ordine seq?

- Două evenimente la **același** moment: tuplurile ar compara apoi **funcțiile** → eroare în Python
- Cu seq, la egalitate câștigă evenimentul programat **primul** (FIFO) – rezultat reproductibil
- Evenimentele simultane sunt frecvente: timpi rotunjiți, programe fixe (ghișeul se deschide la 8:00)
- Ordinea lor poate schimba rezultatul: un client care sosește exact când pleacă altul așteaptă sau nu?
- Regula trebuie **aleasă conștient** și documentată în model

! Atenție la numere reale

În Python 0,1 + 0,2 ≠ 0,3. Nu comparați momente cu ==; folosiți ordinea din heap.

Modelul M/M/1 (1): starea și statisticile

```
from collections import deque

class MM1:
    def __init__(self, sim, lam, mu, rng):
        self.sim, self.lam, self.mu, self.rng = sim, lam, mu, rng
        self.n, self.busy, self.queue = 0, False, deque()
        self.area = self.busy_time = self.last = 0.0
        self.waits = []
        sim.schedule(rng.expovariate(lam), self.arrival)

    def update(self):           # arii sub N(t) și sub „ocupat”
        dt = self.sim.now - self.last
        self.area += self.n * dt
        self.busy_time += self.busy * dt
        self.last = self.sim.now
```

- Stare: **n** clienți, serverul ocupat sau nu, **coada** (momentele de sosire)
- Acumulatori: aria sub N(t), timpul ocupat, lista așteptărilor
- update se apelează la **începutul** fiecărui eveniment, înainte ca starea să se schimbe
- Constructorul programează **prima sosire**

Modelul M/M/1 (2): cele două evenimente

```
def arrival(self):
    self.update()
    self.n += 1
    self.sim.schedule(self.rng.expovariate(self.lam), self.arrival)
    if self.busy:
        self.queue.append(self.sim.now)
    else:
        self.start(self.sim.now)

def start(self, t_arrival):
    self.busy = True
    self.waits.append(self.sim.now - t_arrival)
    self.sim.schedule(self.rng.expovariate(self.mu), self.departure)

def departure(self):
    self.update()
    self.n -= 1
    self.busy = False
    if self.queue:
        self.start(self.queue.popleft())
```

- arrival: programează **următoarea sosire**; apoi coadă sau servire
- start: înregistrează așteptarea și programează **plecarea**
- departure: eliberează serverul; dacă e cineva la rând, începe servirea lui
- Exact tabelul „logica evenimentelor” din ora 1

Rularea și rezultatele

```
import random

sim = Sim()
m = MM1(sim, lam=0.8, mu=1.0, rng=random.Random(11))
sim.run(until=200_000)

L = m.area / sim.now          # medie ponderată în timp
rho = m.busy_time / sim.now   # utilizarea
Wq = sum(m.waits) / len(m.waits)
W = L / m.lam                 # Little
print(L, rho, Wq, W)
```

- random.Random(seed): un generator **propriu**, reproductibil (S2)
- 200 000 de unități de timp ≈ 160 000 de clienți
- L și ρ din **arii**; Wq din lista așteptărilor
- W prin legea lui **Little** – o verificare în plus

Primele evenimente ale unei rulări (λ = 0,8, μ = 1)

Momentul t	Eveniment	N după eveniment	În coadă
0,340	sosire	1	0
0,802	plecare	0	0
1,322	sosire	1	0
2,305	plecare	0	0
2,480	sosire	1	0
2,493	plecare	0	0
2,564	sosire	1	0
2,865	plecare	0	0
4,835	sosire	1	0
5,169	sosire	2	1
5,963	sosire	3	2

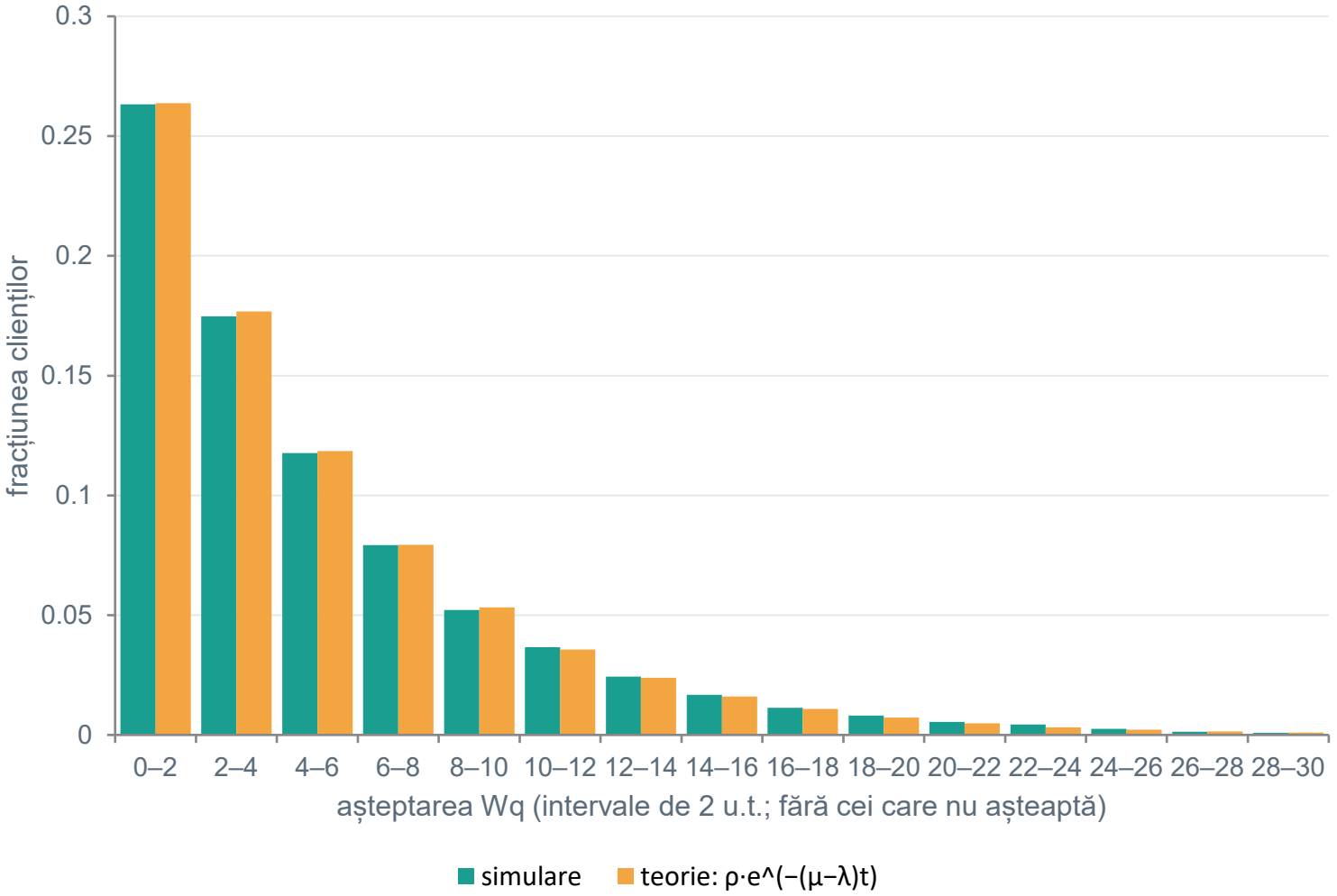
Primii patru clienți găsesc serverul liber; de la t ≈ 4,8 se formează o coadă – începe un „val” de congestie.

Motorul nostru vs formulele M/M/1 (μ = 1)

ρ	L simulat	L teoretic	ρ̂ simulat	W (Little)	W teoretic
0,5	0,99	1,00	0,498	1,98	2,00
0,8	4,05	4,00	0,802	5,06	5,00
0,9	9,50	9,00	0,902	10,56	10,00

- La ρ = 0,5 și 0,8 simularea confirmă formulele la **1–2%**
- La ρ = 0,9 eroarea e mai mare (~5%): clienții consecutivi sunt puternic **corelați**, perioadele de congestie sunt lungi
- Câtă simulare e „suficientă”? Replicări și intervale de încredere – **S6**

Nu doar medii: distribuția așteptării (ρ = 0,8)



- **20%** nu așteaptă deloc (teoretic 1 − ρ = 20%)
- Mediana: 2,36 (teoretic 2,35)
- p95: **14,2** (teoretic 13,9)
- Simularea dă **orice** statistică: percentile, maxime, histograme

Cum verificăm un simulator

- **Urme** pe cazuri mici, comparate cu un calcul de mână (ora 1)
- **Cazuri extreme**: λ foarte mic → nimeni nu așteaptă; timpi constanți → D/D/1, fără cozi
- **Conservare**: sosiri = plecări + clienți rămași; utilizarea ≈ λ/μ
- **Little**: $L = \lambda \cdot W$ pe datele simulate
- **Formule** acolo unde există (M/M/1, M/M/c, M/G/1) – abia apoi trecem la modelul real

✓ **Verificare vs validare**

Verificarea: programul face ce credem noi. **Validarea**: modelul seamănă cu sistemul real (S6).

Cât de rapid este motorul nostru?

≈ 1,0 mil.

evenimente pe secundă în Python pur (măsurat în mediul nostru de test)

0,16 s

pentru 159 070 de evenimente – aproximativ 80 000 de clienți M/M/1

< 1 ms

pentru **o zi întreagă** la bancă: 300 de clienți, ~600 de evenimente

- Răspunsul la întrebarea din S4: o zi la bancă se simulează în mai puțin de o milisecundă
- Simulatoarele industriale (C++, Java) procesează zeci de milioane de evenimente pe secundă

Extensii

Modelul crește, motorul rămâne

1. M/M/c: mai multe servere
2. Priorități: clienți interactivi înaintea celor „batch”
3. Clienți care renunță: anularea evenimentelor
4. Alte extensii: defecțiuni, program de lucru, date reale

M/M/c: câteva linii schimbate

```
class MMc(MM1):                      # c servere, o coadă comună
    def __init__(self, sim, lam, mu, c, rng):
        self.c, self.free = c, c
        super().__init__(sim, lam, mu, rng)

    # update: busy_time += (c - free) * dt

    def arrival(self):
        ...                          # la fel ca la MM1
        if self.free == 0:           # toate serverele ocupate
            self.queue.append(self.sim.now)
        else:
            self.start(self.sim.now)

    def start(self, t_arrival):
        self.free -= 1               # ocupă un server
        ...

    def departure(self):
        ...
        self.free += 1               # eliberează un server
        ...
```

- În loc de busy (da/nu) numărăm **serverele libere**
- Clientul așteaptă doar dacă free == 0
- Utilizarea: timpul ocupat împărțit la **c·T**
- Codul complet e în notebook

M/M/c simulat vs formula Erlang C

λ	c	ρ	Wq simulat	Wq Erlang C	P(așteptare) sim.	P(așteptare) C
1,6	2	0,8	1,782	1,778	0,709	0,711
2,7	3	0,9	2,634	2,724	0,816	0,817
9	10	0,9	0,676	0,669	0,669	0,669

- Aceeași utilizare (0,9) dă așteptări foarte diferite: **2,7** cu 3 servere, **0,67** cu 10 – economia de scară din S4
- P(așteptare) simulat coincide cu formula Erlang C la a treia zecimală
- μ = 1; 400 000 de unități de timp pentru fiecare rând

Priorități: două cozi, alegem mereu întâi clasa 0

```
class Prio(MM1):
    def __init__(self, sim, lam, mu, p0, rng):
        self.p0 = p0                          # fracțiunea clasei 0
        self.qs = [deque(), deque()]
        self.wk = [[], []]                    # așteptările pe clase
        super().__init__(sim, lam, mu, rng)

    def arrival(self):
        ...
        k = 0 if self.rng.random() < self.p0 else 1
        if self.busy: self.qs[k].append(self.sim.now)
        else: self.startk(self.sim.now, k)

    def departure(self):
        ...
        for k in (0, 1):                      # întâi clasa 0
            if self.qs[k]:
                self.startk(self.qs[k].popleft(), k)
                break
```

- Clasa 0: cereri **interactive** (30%); clasa 1: joburi „batch” (70%)
- **Non-preemptiv**: servirea în curs nu se întrerupe
- La fiecare plecare alegem primul client din clasa cea mai importantă
- ρ = 0,8 în total, ca la M/M/1

Priorități: cine câștigă și cine plătește? (ρ = 0,8)

Clasa	Wq simulat	Wq teoretic (Cobham)	p95 simulat
0 – interactive (30%)	1,06	1,05	3,7
1 – batch (70%)	5,14	5,26	18,3
Media ponderată	3,92	4 (ca FIFO)	–

- Cererile interactive așteaptă de ~4 ori **mai puțin** decât la FIFO (1,05 vs 4)
- Joburile batch plătesc: așteptarea lor crește de la 4 la ~5,2
- Media ponderată rămâne aceeași: prioritățile **redistribuie** așteptarea, nu o elimină (legea conservării lui Kleinrock)

Clienți care renunță: anularea „leneșă” a evenimentelor

```
def arrival(self):
    ...
    c = [self.sim.now, "așteaptă"]
    if self.busy:
        self.queue.append(c)
        self.sim.schedule(self.rng.expovariate(1 / self.patience),
                          self.renege, c)
    else:
        self.start(c)

def renege(self, c):
    if c[1] == "așteaptă":        # încă în coadă? pleacă
        c[1] = "plecat"
        ...                         # n -= 1, lost += 1

def departure(self):
    ...
    while self.queue and self.queue[0][1] == "plecat":
        self.queue.popleft()        # ștergere „leneșă”
```

- La sosire programăm și evenimentul **Renunță**, după o răbdare aleatoare
- Dacă clientul a fost deja servit, evenimentul **nu mai face nimic**
- Clienții plecați sunt scoși din coadă abia când ajung în față
- $\lambda = 1{,}2 > \mu = 1$, răbdare medie 5: **30%** renunță, Wq = 1,48

Clienți care renunță: ce ne spune simularea (λ = 1,2, μ = 1)

Mărime	Valoare simulată
Fracțiunea clienților care renunță	29,9%
Clienți serviți pe unitatea de timp (throughput)	0,839
Utilizarea serverului	0,842
Așteptarea medie a celor serviți	1,48
Numărul mediu de clienți în sistem	2,62

- Fără renunțări, la ρ = 1,2 coada ar crește **nelimitat** (S4): modelul ar fi instabil
- Renunțările „reglează” singure sistemul: pleacă exact cât e nevoie ca serverul să nu fie depășit
- Pentru afaceri, 30% clienți pierduți e un cost – simularea îl **cuantifică** (câți casieri ar reduce pierderea sub 5%?)

Alte extensii, fără să atingem motorul

1 Capacitate finită

La sosire: dacă sistemul are deja K clienți, clientul este refuzat și numărat ca pierdut (M/M/1/K).

2 Defecțiuni

Evenimente Defect și Reparat; în timpul reparației serverul nu servește (timp mediu între defecțiuni, MTBF).

3 Program de lucru

Evenimente Deschide și Închide pentru ghișee; numărul de servere variază în timpul zilei.

4 Rețele

La plecare, clientul este trimis la alt nod, după probabilitățile de rutare (Jackson, S4).

5 Rate variabile

λ(t) diferit dimineața și la prânz: sosirile se generează din rata curentă (S6).

6 Date reale

Sosirile și duratele citite dintr-un jurnal: simulare condusă de date (trace-driven).

Trei abordări

Evenimente, activități, procese

1. Programarea evenimentelor (event scheduling)
2. Scanarea activităților și metoda în trei faze
3. Interacțiunea proceselor (process interaction)
4. Istoric: de la GPSS și Simula la SimPy

Abordarea 1: programarea evenimentelor

Event scheduling

Modelul este descris ca o listă de **tipuri de evenimente**; fiecare are o rutină care schimbă starea și **programează** evenimente viitoare. Ceasul sare de la un eveniment la următorul.

- Exact ce am făcut până acum: Sosire, Plecare, Renunță
- Foarte eficientă și ușor de implementat
- Logica unei entități este **împrăștiată** în mai multe rutine – modelele mari devin greu de citit

Abordarea 2: scanarea activităților (metoda în trei faze)

Activity scanning / three-phase (Tocher, 1963)

Evenimentele **B** (bound) au loc la momente cunoscute: sfârșitul unei serviri, o sosire. Evenimentele **C** (conditional) au loc când o **condiție** devine adevărată: „coada nu e goală **și** serverul e liber”.

- Faza A: avansăm ceasul la următorul eveniment B
- Faza B: executăm toate evenimentele B de la acest moment
- Faza C: scanăm toate condițiile C până nu mai pornește nicio activitate

Abordarea 3: interacțiunea proceselor

Process interaction

Descriem **viața unei entități** ca un program secvențial: „sosesc, cer serverul, **aștept** până îl primesc, sunt servit, plec”. Motorul suspendă și reia automat aceste programe.

- Codul arată ca **povestea** clientului – ușor de citit și de verificat
- Folosită de GPSS, Simula, SimPy, Arena, AnyLogic
- Dedesubt există tot o **listă de evenimente**: fiecare „aștept” devine un eveniment programat

Cele trei abordări pe scurt

	Evenimente	Activități (3 faze)	Procese
Descriem	tipuri de evenimente	activități cu condiții de start	viața fiecărei entități
Punctul forte	viteză, control total	resurse partajate complicate	**lizibilitate**
Punctul slab	logică împrăștiată	scanarea condițiilor costă	suspendarea proceselor costă puțin
Exemple	motorul nostru, SIMSCRIPT, ns-3	CSL, unele unelte britanice	SimPy, Simula, GPSS, Arena

- Toate trei produc **aceleași rezultate** pentru același model – diferă doar modul în care scriem modelul
- În practică: procese pentru modelare rapidă, evenimente pentru performanță maximă (simulatoare de rețele)

Același client, scris ca proces (previzualizare SimPy)

```
def client(env, server, mu):
    t0 = env.now
    with server.request() as cerere:
        yield cerere                          # aștept serverul
        yield env.timeout(random.expovariate(mu))   # servire
    timpi.append(env.now - t0)                # am plecat

def sursa(env, server, lam, mu):
    while True:
        yield env.timeout(random.expovariate(lam))
        env.process(client(env, server, mu))
```

- yield = „aici procesul se oprește și așteaptă”
- Nu mai scriem explicit Sosire și Plecare: le generează **SimPy**
- Generatoarele Python (yield) fac posibilă această abordare
- Detalii în ora 3

O scurtă istorie a limbajelor de simulare

1961 — **GPSS** (G. Gordon, IBM): blocuri prin care curg tranzacții

1963 — **SIMSCRIPT** (Markowitz, RAND); metoda în trei faze (Tocher)

1967 — **Simula 67** (Dahl, Nygaard): primul limbaj **orientat pe obiecte** – născut din simulare!

1983–93 — **SIMAN**, apoi **Arena**: modelare grafică pentru industrie

2000 — **AnyLogic**: evenimente discrete + agenți + dinamica sistemelor

2002 — **SimPy**: procese cu generatoare Python, open source

Clasele și obiectele din Java, C++ și Python vin direct din nevoia de a descrie entitățile unei simulări.

Contează implementarea: structura listei de evenimente

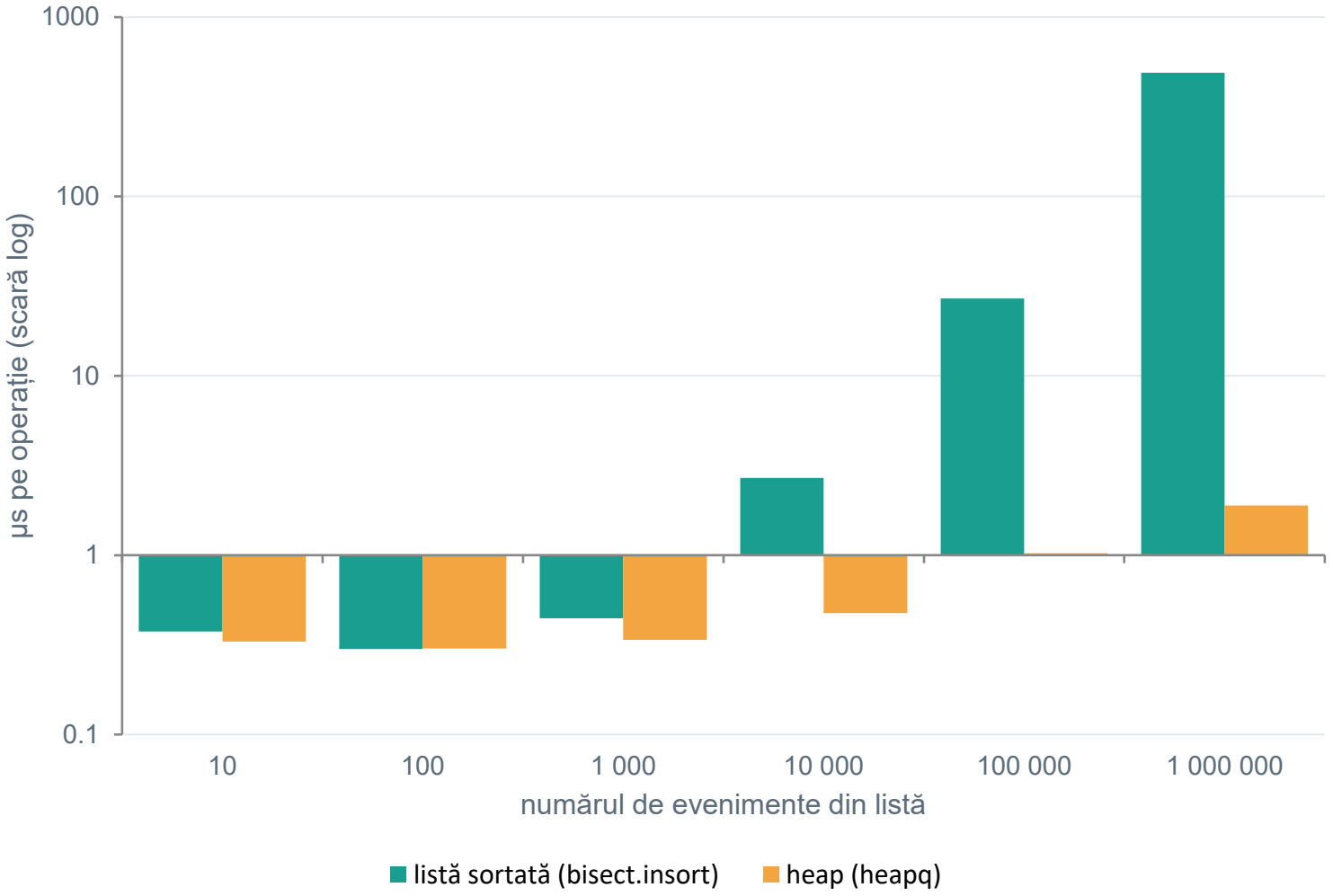
- **Listă sortată**: inserare O(n), extragere O(1) – bună doar pentru liste mici
- **Heap binar**: inserare și extragere O(log n) – alegerea implicită (heapq)
- **Calendar queue** (Brown, 1988): „găleți” pe intervale de timp, O(1) în medie
- **Pairing heap**, arbori splay: variante rapide în practică (preferatele lui Cernazean)
- Multe simulatoare petrec **cea mai mare parte** a timpului în operațiile pe lista de evenimente (Cambridge)

H

Modelul „hold”

Test standard: scoatem minimul și inserăm un eveniment nou, de milioane de ori, cu n evenimente în listă.

Listă sortată vs heap: microsecunde pe operație (modelul „hold”)



- Până la ~1 000 de evenimente: **practic egale**
- La 100 000: listă 27 µs vs heap 1,0 µs
- La 1 milion: **de ~259 de ori** mai lent cu lista
- Simulări mari = multe evenimente în așteptare

Capcane frecvente

- **Statistici actualizate prea târziu**: update() trebuie apelat **înainte** de a schimba starea
- **Evenimente simultane** fără o regulă clară de ordine
- **Același generator** pentru toate sursele: schimbi servirea și se schimbă și sosirile
- **Medii simple** în loc de medii ponderate în timp pentru L sau ρ
- **Starea inițială goală**: primele ore nu sunt reprezentative (încălzire, S6)
- O singură rulare raportată ca „rezultatul” simulării

R

Fluxuri separate

rng_sosiri = Random(1), rng_servire = Random(2): comparăm variante cu **aceleași** sosiri (S6).

DES în informatică

1 Rețele

ns-3, OMNeT++: fiecare pachet trimis sau primit este un eveniment; se testează protocoale noi înainte de implementare.

2 Arhitecturi

gem5: procesoare și memorii simulate la nivel de ciclu, pentru a evalua un cip înainte de fabricare.

3 Cloud

CloudSim: centre de date, mașini virtuale și politici de alocare, fără să plătim hardware-ul.

4 Planificatoare

Simulatoare pentru Kubernetes și Slurm: testăm politici de programare a joburilor pe jurnale reale.

5 Sisteme distribuite

Testare deterministă (FoundationDB): tot sistemul rulează într-un simulator cu ceas propriu.

6 Jocuri, robotică

Motoare cu evenimente pentru logica jocului; simulatoare de senzori și comenzi pentru roboți.

Verificare

? O simulare de rețea are în medie **1 milion** de evenimente în lista evenimentelor viitoare. Ce structură de date alegem?

A o listă sortată, pentru că extragerea minimului e O(1)

B o listă nesortată, parcursă la fiecare pas

C un heap binar sau o structură specializată (calendar queue)

D un dicționar indexat după moment

Verificare

? Ce abordare folosește SimPy, în care viața unui client este scrisă ca o funcție cu yield?

A programarea evenimentelor

B scanarea activităților (trei faze)

C interacțiunea proceselor

D simularea cu pas fix

Verificare

? În modelul MM1, un client sosește când serverul este **ocupat**. Ce eveniment nou este programat în acțiunea arrival?

A doar sosirea următorului client

B sosirea următoare și plecarea acestui client

C doar plecarea acestui client

D niciun eveniment

Verificare

? Pentru ρ = 0,9, simulatorul dă W = 10,56, iar formula M/M/1 dă 10. Care este cea mai probabilă explicație?

A motorul are o eroare de logică

B variabilitatea statistică: la ρ mare clienții sunt puternic corelați și e nevoie de rulări mai lungi sau mai multe

C formula M/M/1 nu este valabilă la ρ = 0,9

D heap-ul ordonează greșit evenimentele

Rezumatul orei 2

- ✓ Motorul: **heap** (heapq) + bucla „extrage minimul, mută ceasul, execută” – ~10 linii
- ✓ Modelul M/M/1: evenimentele **Sosire** și **Plecare**, statistici din arii; reproduce formulele din S4
- ✓ Extensii fără a atinge motorul: **M/M/c**, **priorități**, **renunțări** (anulare leneșă)
- ✓ Trei abordări: **evenimente**, activități (trei faze), **procese** (SimPy, Simula)
- ✓ Implementarea contează: heap vs listă; ordinea evenimentelor simultane; fluxuri aleatoare separate

Ora 3

SimPy și aplicații

1. SimPy: procese, resurse, timeout-uri
2. Intersecția cu semafor (Cernazean)
3. Linia de producție și unitatea de primiri urgențe
4. DES în industrie: unelte, digital twins, simulare paralelă

Unde am rămas

- ✓ Motorul nostru: **heap** + buclă; modelul M/M/1 reproduce formulele din S4
- ✓ Extensii ușoare: M/M/c, priorități, clienți care renunță
- ✓ Trei abordări: evenimente, activități, **procese**
- ✓ Acum: abordarea pe procese, cu o bibliotecă matură – **SimPy**

SimPy

SimPy – simulare cu evenimente discrete în Python

O bibliotecă open source (din 2002) în care fiecare entitate activă este un **proces**: o funcție generator Python care, cu yield, **așteaptă** evenimente – trecerea timpului, eliberarea unei resurse, sosirea unui mesaj.

- **Environment**: ceasul simulat și lista evenimentelor viitoare – motorul
- **Process**: viața unei entități (client, mașină, pacient), scrisă ca o poveste secvențială
- **Resource**: servere cu capacitate limitată și coadă automată (FIFO sau cu priorități)

Piesele SimPy

1 env.timeout(d)

Un eveniment care are loc după d unități de timp: „lucrez” sau „mă deplasez” d minute.

2 env.process(g)

Pornește un proces nou din generatorul g; de exemplu, un client nou la fiecare sosire.

3 Resource

c servere identice; request() intră la coadă, iar ieșirea din blocul with eliberează serverul.

4 PriorityResource

La fel, dar coada este ordonată după prioritate: pacienții urgenți trec înainte.

5 Container

O cantitate continuă sau în vrac: combustibil, biciclete într-o stație (laboratorul).

6 Store

Obiecte distincte într-un depozit limitat: piese într-un buffer, mesaje într-o coadă.

Primul proces: un ceas care bate

```
import simpy

def ceas(env, nume, perioada):
    while True:
        print(f"{env.now:5.1f}  {nume}")
        yield env.timeout(perioada)     # doarme „perioada”

env = simpy.Environment()
env.process(ceas(env, "rapid", 0.5))
env.process(ceas(env, "lent", 1.0))
env.run(until=2)

#   0.0  rapid
#   0.0  lent
#   0.5  rapid
#   1.0  rapid
#   1.0  lent
#   1.5  rapid
```

- Două procese rulează „în paralel” în timpul simulat
- yield env.timeout(...) **suspendă** procesul; SimPy îl reia la momentul potrivit
- Nu există fire de execuție: totul rulează secvențial, dintr-o listă de evenimente
- La t = 1,0 ambele procese se trezesc: ordinea = ordinea programării

Coada M/M/1 în SimPy

```
def simpy_mm1(lam, mu, T, seed):
    rng = random.Random(seed)
    env = simpy.Environment()
    server = simpy.Resource(env, capacity=1)
    times = []

    def customer(env):
        t0 = env.now
        with server.request() as req:
            yield req                                  # așteaptă serverul
            yield env.timeout(rng.expovariate(mu))     # servirea
        times.append(env.now - t0)

    def source(env):
        while True:
            yield env.timeout(rng.expovariate(lam))
            env.process(customer(env))

    env.process(source(env))
    env.run(until=T)
    return sum(times) / len(times), len(times)
```

- source: generează clienți la intervale exponențiale
- customer: povestea unui client – **cere**, **așteaptă**, e **servit**, pleacă
- with ... as req: serverul se eliberează automat la ieșirea din bloc
- Fără evenimente Sosire / Plecare scrise de noi

SimPy vs motorul nostru vs formula (M/M/1, ρ = 0,8)

Varianta	W (timp în sistem)	Clienți	Timp de execuție
Formula M/M/1: 1/(μ − λ)	5,00	–	–
Motorul nostru (evenimente)	5,060	≈ 160 364	0,24 s
SimPy (procese)	5,048	160 364	1,58 s

- Ambele simulatoare dau W ≈ 5,05, la **1%** de formulă – validare reciprocă
- SimPy este de ~7 ori mai lent: generatoarele și obiectele-eveniment costă
- Pentru majoritatea studiilor viteza SimPy e suficientă; contează mai mult **claritatea** modelului

Ce face SimPy în spatele scenei

- Fiecare yield întoarce un **obiect eveniment** (timeout, request, get, put)
- SimPy pune evenimentul în **lista evenimentelor viitoare** (un heap, ca la noi) și suspendă generatorul
- Când evenimentul are loc, SimPy **reia** generatorul exact după yield
- request() pe o resursă ocupată: evenimentul are loc abia când resursa se eliberează
- env.run(until=T) este bucla principală: extrage, avansează ceasul, execută – identic cu ora 2

Generatoare Python

O funcție cu yield își **păstrează starea** între apeluri – exact ce trebuie pentru a „pune pe pauză” un client care așteaptă.

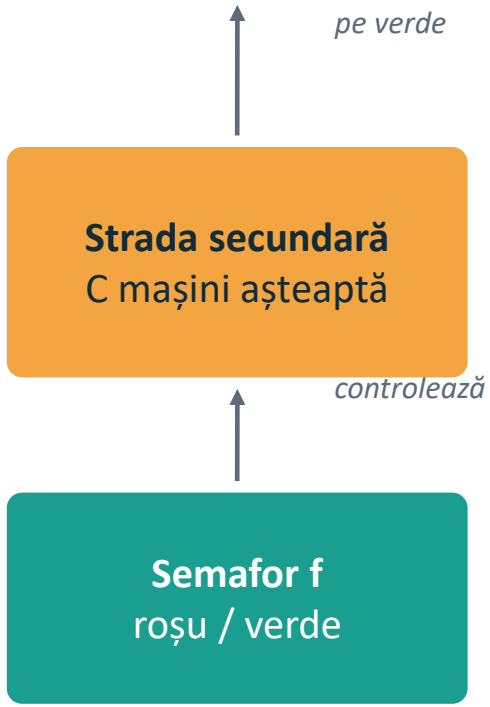
Aplicația 1 (Cernazean L7)

Intersecția cu semafor

1. O stradă secundară cu semafor la cerere
2. Stare, evenimente, parametri
3. Urma evenimentelor și o erată
4. Efectul latenței semaforului asupra așteptării

Modelul intersecției

Strada principală – are verde, cu excepția perioadelor cerute de strada secundară



Cum funcționează

Un **senzor** detectează prima mașină care ajunge la roșu.

După o latență **a**, semaforul trece pe verde.

Verdele durează **b** secunde pentru fiecare mașină care aștepta (C·b), apoi revine la roșu.

Mașinile care sosesc pe verde trec direct.

Stare, evenimente și parametri

Element	Descriere
Stare	culoarea semaforului $f \in \{\text{roșu}, \text{verde}\}$; numărul de mașini care așteaptă $C \in \mathbb{N}$
Eveniment CAR	sosește o mașină pe strada secundară (**exogen** – din date sau pseudo-aleator)
Eveniment R2G	semaforul trece din roșu în verde; programează G2R după $C \cdot b$
Eveniment G2R	semaforul trece din verde în roșu
Parametri	**a** = latența până la verde (30 s); **b** = durata verdelui pentru fiecare mașină (10 s)

- Lista evenimentelor viitoare conține la început **toate** sosirile de mașini (evenimente exogene)
- Evenimentele R2G și G2R sunt **endogene**: le programează acțiunile modelului

Acțiunile evenimentelor (cu motorul din ora 2)

```
def CAR():                              # sosește o mașină
    if s.f == "verde":
        waits.append(0)                 # trece imediat
    else:
        if s.C == 0:                    # prima mașină: senzorul cere verde
            sim.schedule(a, R2G)
        s.C += 1                        # (lipsea în codul original)
        queue.append(sim.now)

def R2G():                              # roșu → verde
    s.f = "verde"
    sim.schedule(s.C * b, G2R)          # verde cât să treacă C mașini
    waits.extend(sim.now - t for t in queue)
    queue.clear()
    s.C = 0

def G2R():                              # verde → roșu
    s.f = "roșu"
```

- Aceeași clasă Sim din ora 2 – motorul nu se schimbă
- Prima mașină la roșu programează R2G
- R2G programează G2R și eliberează coada
- Colectăm așteptarea fiecărei mașini

Urma evenimentelor (a = 30, b = 10), ca în cursul lui Cernazean

t	Eveniment	C	f	Lista evenimentelor viitoare
0	start	0	roșu	CAR(10) CAR(25) CAR(35) CAR(60) CAR(75)
10	CAR	1	roșu	CAR(25) CAR(35) R2G(40) CAR(60) CAR(75)
25	CAR	2	roșu	CAR(35) R2G(40) CAR(60) CAR(75)
35	CAR	3	roșu	R2G(40) CAR(60) CAR(75)
40	R2G	0	verde	CAR(60) G2R(70) CAR(75)
60	CAR	0	verde	G2R(70) CAR(75)
70	G2R	0	roșu	CAR(75)
75	CAR	1	roșu	R2G(105)
105	R2G	0	verde	G2R(115)
115	G2R	0	roșu	Ø

Mașini la 10, 25, 35, 60 și 75 s. Așteptări: 30, 15, 5, 0 și 30 s (medie 16 s).

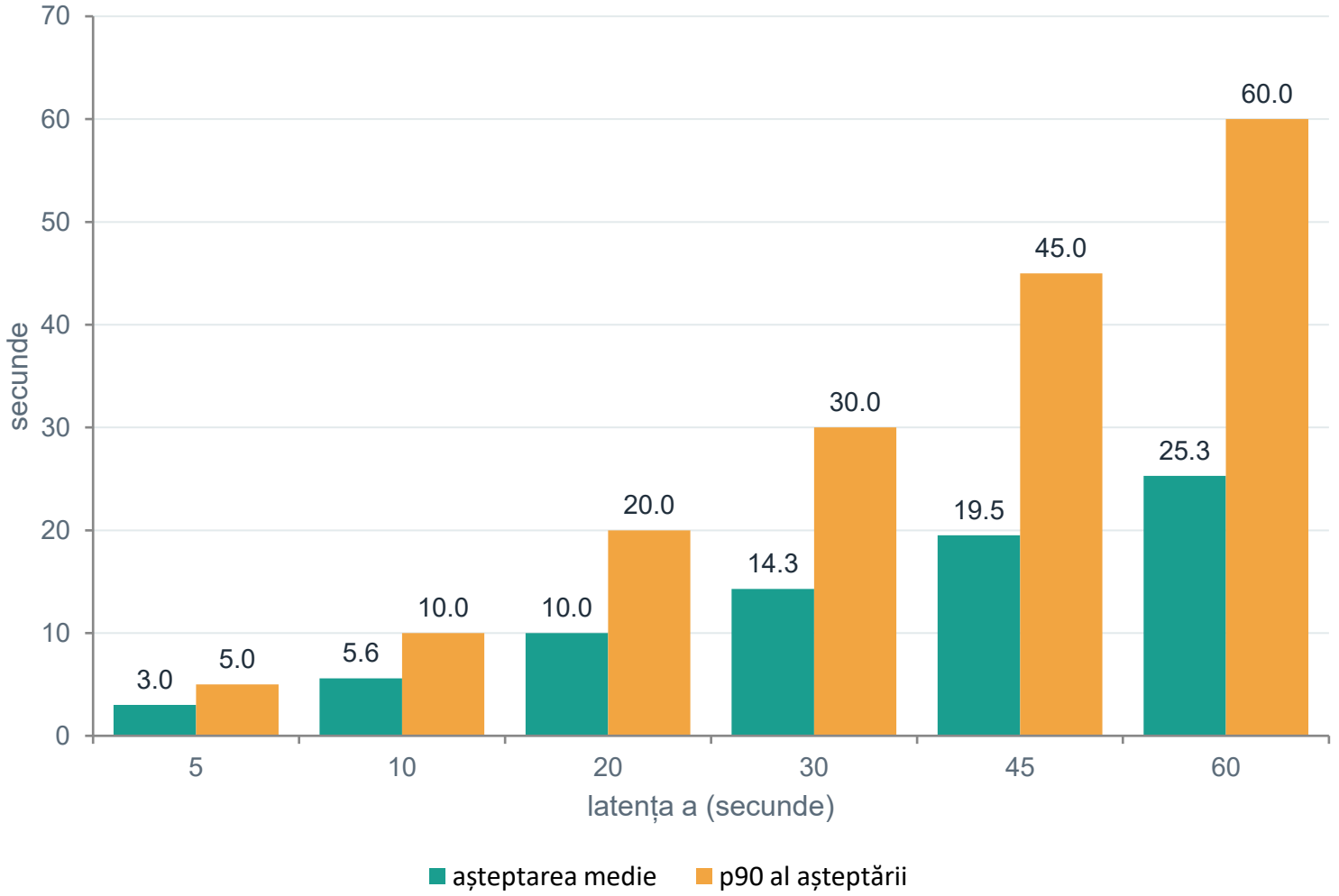
O eroare și o lecție despre verificare

- În codul original, ramura „prima mașină la roșu” programa R2G, dar **nu** incrementa C
- Cu acel cod, la t = 10 am avea C = 0, iar la t = 25 s-ar programa **încă un** R2G
- Tabelul din curs arată însă C = 1 la t = 10: tabelul a fost făcut „de mână”, corect
- Lecția: **urma de mână** și **codul** trebuie comparate – exact verificarea din ora 2
- Varianta corectă: C += 1 pentru orice mașină care sosește pe roșu

✓ **Test automat**

Rulăm codul pe sosirile 10, 25, 35, 60, 75 și comparăm cu tabelul. Diferența apare imediat.

Latența semaforului a vs așteptarea mașinilor (b = 10 s)



- Sosiri Poisson: o mașină la **20 s** în medie, 8 ore
- ~33% dintre mașini prind verde și nu așteaptă deloc
- Așteptarea medie crește aproape liniar cu a
- p90 = a: prima mașină din fiecare ciclu așteaptă exact a

Complicații realiste (Cernazean)

- O **limită** pentru durata verdelui: strada principală nu poate aștepta oricât
- Un timp **realist de trecere** pentru fiecare mașină (demaraj, distanța dintre mașini)
- O **trecere de pietoni** cu propriul buton și propriul ciclu
- **Sincronizarea** semafoarelor vecine („undă verde”) – o rețea de intersecții
- Trafic variabil în timpul zilei: ora de vârf dimineața și seara (S6)

+ Ce se schimbă în cod?

Fiecare complicație = un parametru sau un tip nou de eveniment. **Motorul rămâne același.**

Aplicația 2

O linie de producție

1. Trei stații în serie, cu buffere între ele
2. Blocare și „înfometare”
3. Throughput-ul în funcție de mărimea bufferelor
4. Legătura cu gâtul de sticlă și cu VUT (S4)

Linia: stații în serie cu buffere limitate



Blocare: stația 1 a terminat o piesă, dar bufferul din fața stației 2 e plin – stația 1 **stă** cu piesa în mână.
Înfometare: stația 3 e liberă, dar bufferul din fața ei e gol – stația 3 **așteaptă** piese.
Stația 2 este **gâtul de sticlă** (1,1 min/piesă): throughput-ul nu poate depăși 1/1,1 ≈ 0,91 piese/min.

Linia în SimPy: Store = buffer limitat

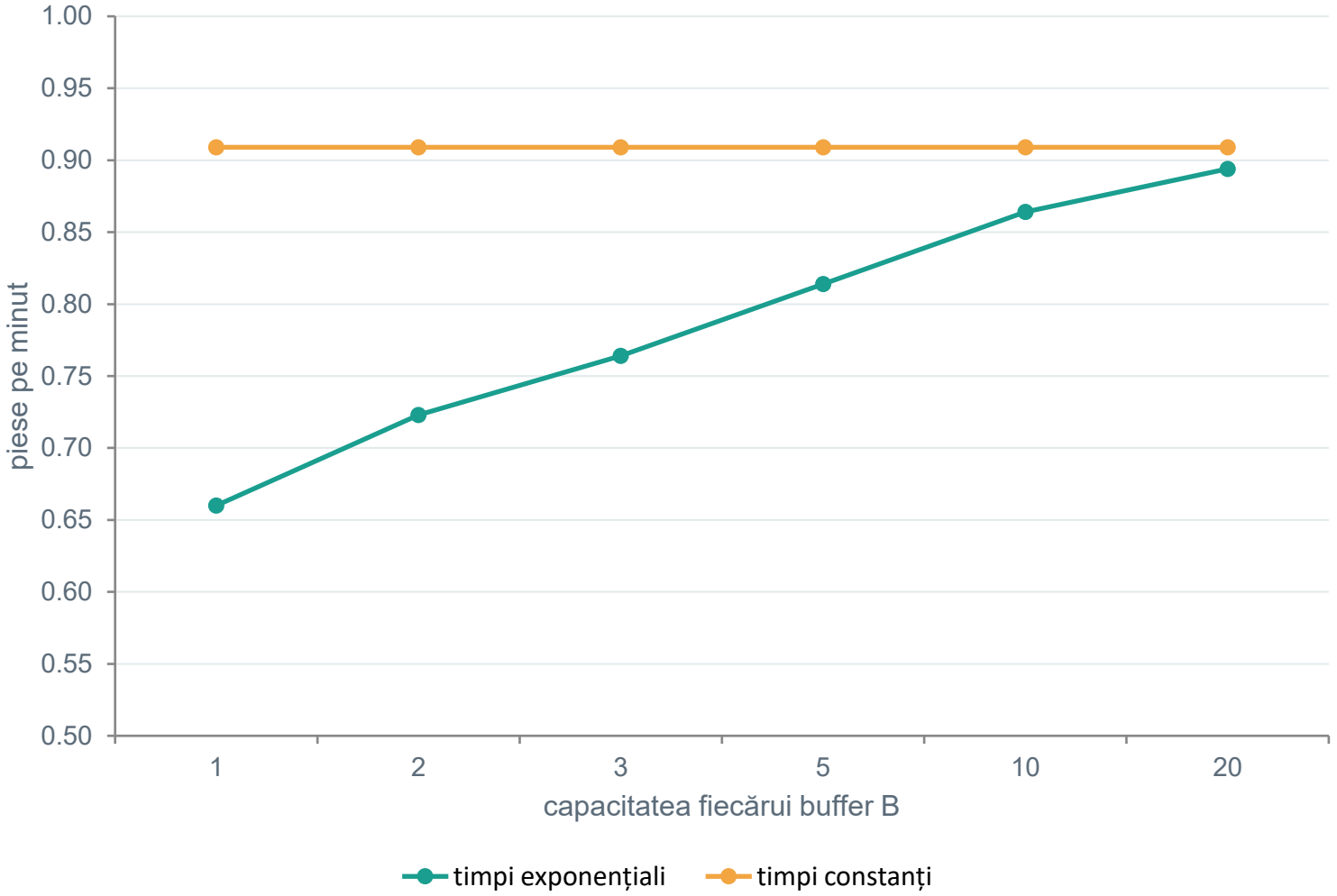
```
def line(B, T=50_000, seed=5, means=(1.0, 1.1, 0.9)):
    rng = random.Random(seed)
    env = simpy.Environment()
    buf = [simpy.Store(env, capacity=B) for _ in range(2)]
    done = [0]

    def station(env, k):
        while True:
            if k > 0:
                yield buf[k - 1].get()     # ia o piesă (sau „flămânzește”)
            yield env.timeout(rng.expovariate(1 / means[k]))
            if k < 2:
                yield buf[k].put("piesă")  # predă piesa (sau „se blochează”)
            else:
                done[0] += 1

    for k in range(3):
        env.process(station(env, k))
    env.run(until=T)
    return done[0] / T                     # piese pe minut
```

- get() pe un buffer gol **așteaptă** – înfometare
- put() pe un buffer plin **așteaptă** – blocare
- Fiecare stație este un proces cu o buclă infinită
- 50 000 de minute simulate pentru fiecare B

Throughput-ul liniei în funcție de mărimea bufferelor



- Timpi exponențiali, B = 1: **0,66** piese/min – doar 73% din maxim
- B = 20: 0,89 – aproape de limita 0,91
- Timpi constanți: **0,91** chiar cu B = 1
- Bufferele plătesc prețul **variabilității**

Lecțiile liniei de producție

- Throughput-ul este limitat de **gâtul de sticlă** (stația cea mai lentă) – ca în rețelele din S4
- Variabilitatea „fură” capacitate: fără buffere, stațiile se blochează reciproc (**VUT**, S4)
- Două pârghii: **reducem variabilitatea** (standardizare) sau **adăugăm buffere** (cost: stocuri, spațiu)
- Bufferul contează mai ales **în fața** gâtului de sticlă: stația 2 nu trebuie să stea niciodată
- Teoria constrângerilor (Goldratt), Kanban, Lean: aceleași idei, în limbajul managementului

? Formulă sau simulare?

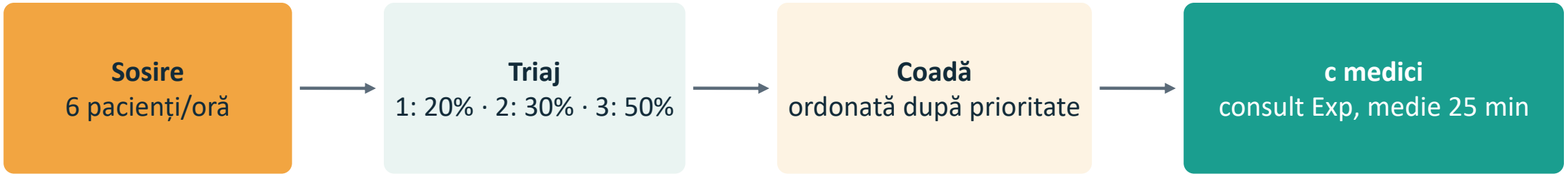
Liniile cu buffere finite **nu** au formule simple – de aceea fabricile folosesc simularea.

Aplicația 3

Unitatea de primiri urgențe

1. Triaj cu trei niveluri de prioritate
2. PriorityResource în SimPy
3. Câți medici? Întrebări de tip „ce-ar fi dacă”
4. Și o parcare (Holban): când modelul ne spune că datele sunt greșite

Fluxul pacienților



Priorități: 1 = urgență imediată, 2 = urgent, 3 = poate aștepta. Un pacient cu prioritate mai mare trece înaintea celor din coadă, dar **nu întrerupe** o consultație în curs (non-preemptiv).
Încărcarea: a = 6 × 25/60 = 2,5 medici ocupați în medie → cu 3 medici ρ = 0,83; cu 4 medici ρ = 0,63.

PriorityResource: coada ordonată după urgență

```
def ed(c, T=1_000_000, seed=9, lam=6/60, mean_consult=25.0,
       probs=(0.2, 0.3, 0.5)):
    rng = random.Random(seed)
    env = simpy.Environment()
    doctors = simpy.PriorityResource(env, capacity=c)
    waits = {1: [], 2: [], 3: []}

    def patient(env, prio):
        t0 = env.now
        with doctors.request(priority=prio) as req:   # 1 = cea mai urgentă
            yield req
            waits[prio].append(env.now - t0)
            yield env.timeout(rng.expovariate(1 / mean_consult))

    def arrivals(env):
        while True:
            yield env.timeout(rng.expovariate(lam))
            u = rng.random()
            prio = 1 if u < probs[0] else 2 if u < probs[0] + probs[1] else 3
            env.process(patient(env, prio))

    env.process(arrivals(env))
    env.run(until=T)
```

- Singura diferență față de M/M/c: **PriorityResource** și priority=prio
- Număr mai mic = prioritate mai mare
- 10^6 minute ≈ 1,9 ani simulați, ~100 000 de pacienți
- Rulăm pentru c = 3 și c = 4

Așteptarea până la medic (minute)

Prioritate	3 medici: medie	3 medici: p90	4 medici: medie	4 medici: p90
1 – urgență imediată	7,0	20	2,3	8
2 – urgent	12,3	35	3,4	12
3 – poate aștepta	60,0	172	7,9	28

- Cu 3 medici (ρ = 0,83) pacienții de prioritate 3 așteaptă în medie **o oră**, iar 10% dintre ei aproape **3 ore**
- Al patrulea medic reduce așteptarea medie a priorității 3 de la 60 la **8** minute
- Chiar și urgențele imediate așteaptă ~7 min cu 3 medici: nu se întrerupe o consultație în curs

Întrebări de tip „ce-ar fi dacă” pentru management

- Câți medici pe fiecare **tură**, dacă sosirile au vârf seara? (rate variabile, S6)
- Merită un **medic dedicat** priorității 3 (fast track) în loc de o coadă comună?
- Ce se întâmplă dacă un **laborator** (analize) devine gâtul de sticlă?
- Cât de sensibil e rezultatul la durata consultației (25 vs 30 min)? – analiză de **sensibilitate**
- Modelele reale de urgențe sunt folosite în spitale pentru planificarea personalului și a spațiilor

Atenție

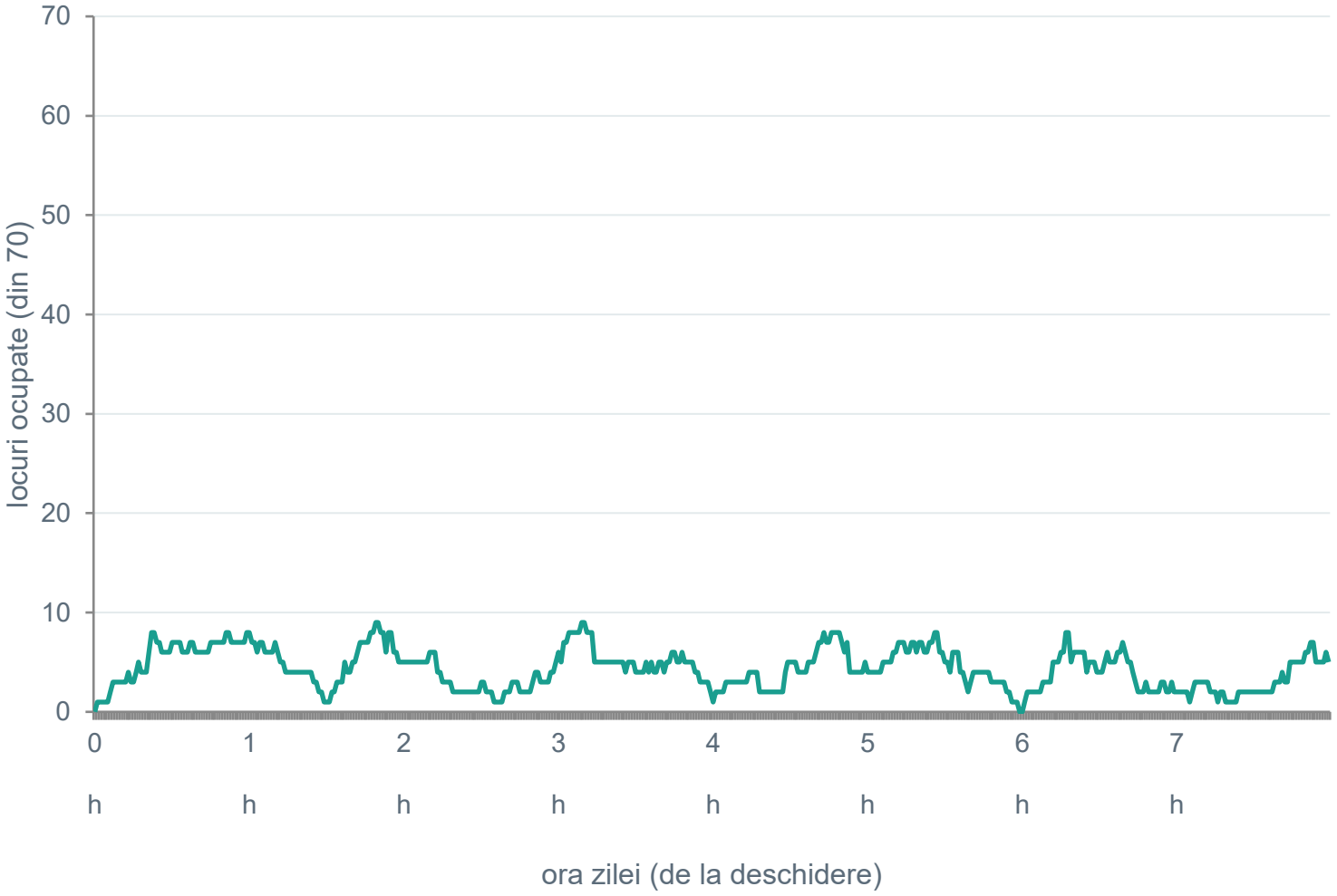
Priorități + ρ mare: media arată bine, dar pacienții de prioritate mică pot aștepta **foarte** mult. Raportați percentile pe clase.

Parcarea cu 70 de locuri (Holban, studiul de caz 3)

	Datele din curs	Varianta aglomerată
Sosiri (exponențiale)	o mașină la 240 s	o mașină la 40 s
Staționare (normală)	medie 1 000 s, abatere 300 s	medie 3 000 s, abatere 900 s
Încărcarea oferită a = λ·s	4,2 locuri ocupate în medie	75 locuri ocupate în medie
Rezultatul simulării	ocupare medie 4,3 (6%), maxim 9; 0 refuzați în 8 ore	**12,8%** refuzați (Erlang B: 13,1%)

- Cu datele din curs, parcarea e aproape **goală**: un calcul de 10 secunde (a = λ·s ≈ 4) ar fi arătat asta înainte de simulare
- Varianta aglomerată confirmă **Erlang B** din S4, chiar dacă staționarea nu e exponențială (Erlang B depinde doar de medie)

Parcarea cu datele din curs: o zi de 8 ore



- Maxim **9** locuri ocupate din 70
- Modelul „funcționează”, dar răspunsul nu are sens pentru un proiect real
- **Validarea** începe cu verificarea ordinelor de mărime (S6)
- Little: $L = \lambda W = (1/240)\cdot 1\,000 \approx 4$ mașini

DES la scară mare

Industrie, digital twins, simulare paralelă

1. Unelte industriale
2. DES ca nucleu al digital twin-urilor de fabrici
3. Simularea paralelă: cauzalitate, conservativ vs optimist

Unelte industriale de simulare

1 Arena (Rockwell)

Modelare grafică prin blocuri (moștenitorul SIMAN); foarte folosită în producție și logistică.

2 AnyLogic

Combină evenimente discrete, agenți și dinamica sistemelor în același model (S9).

3 Simio, FlexSim

Modele orientate pe obiecte, cu animație 3D; planificare în fabrici și depozite.

4 Plant Simulation (Siemens)

Linii de producție și fabrici întregi; parte din ecosistemele de digital twin.

5 SimEvents (MATLAB)

Evenimente discrete integrate cu modele continue în Simulink (sisteme hibride).

6 Open source

SimPy, Salabim, JaamSim, OMNeT++ și ns-3 pentru rețele: gratuite, bune pentru cercetare.

DES ca nucleu al digital twin-urilor de fabrici

- Un **digital twin** = un model al sistemului real, actualizat continuu cu **date din producție**
- Starea inițială a simulării = **starea curentă** a fabricii (comenzi, mașini, stocuri)
- Simulăm **înainte** următoarele ore sau zile: „dacă primim comanda X, când o livrăm?”
- Comparăm variante de planificare în câteva secunde, înainte de a le aplica în realitate
- Aceeași idee în logistică, aeroporturi, spitale, centre de date (S10)

DT **Diferența față de azi**

Nu mai pornim din starea „goală”: pornim din starea **reală**, citită din senzori și din sistemele informatice.

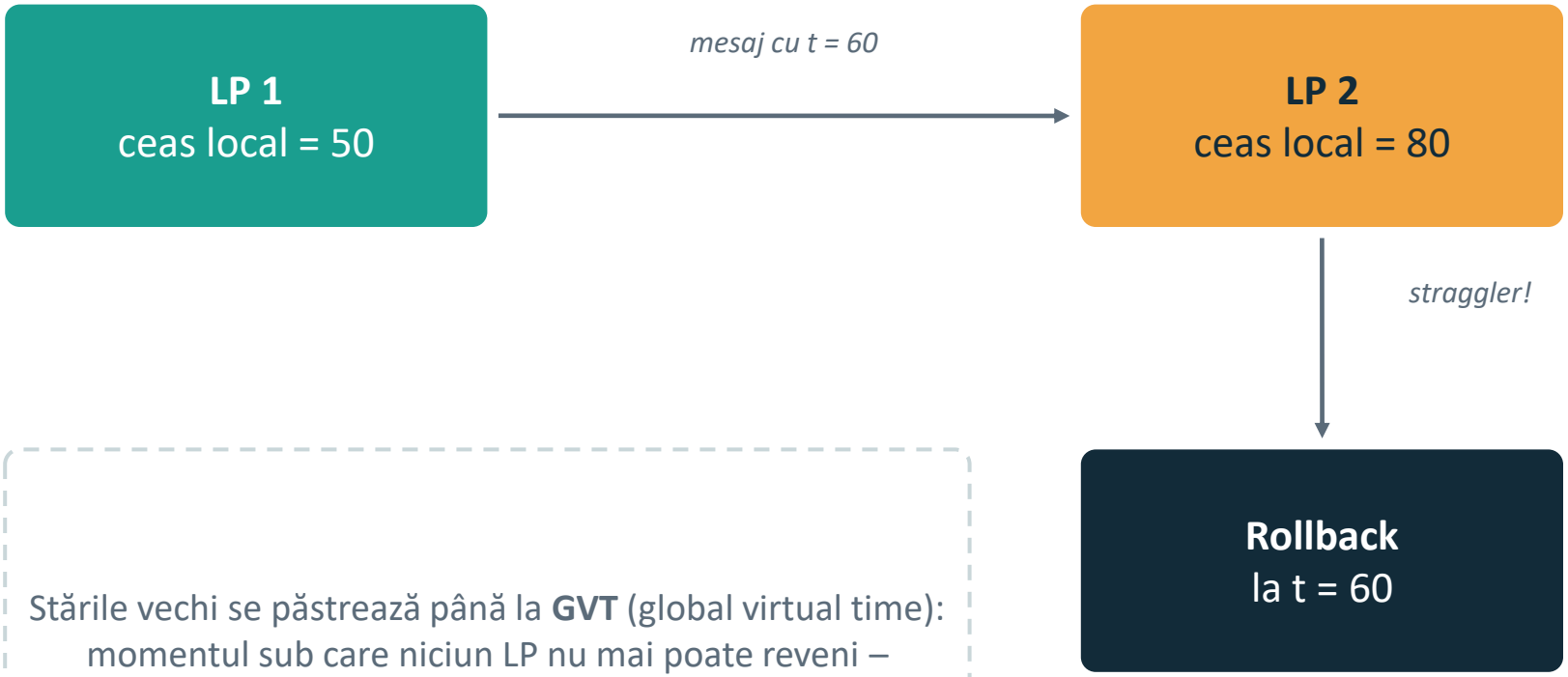
Simularea paralelă cu evenimente discrete (PDES)

Parallel discrete-event simulation

Modelul este împărțit în **procese logice** (LP), fiecare cu propriul ceas și propria listă de evenimente, care își trimit **mesaje cu marcaj de timp**. Problema: un LP nu are voie să proceseze un eveniment dacă ar putea primi ulterior un mesaj din **trecutul** lui.

- **Conservativ** (Chandy–Misra–Bryant, 1979): un LP avansează doar când e **sigur**; mesaje „nule” și lookahead evită blocajele
- **Optimist** (Time Warp, Jefferson, 1985): avansează oricum; la un mesaj întârziat face **rollback** și trimite anti-mesaje
- Folosită pentru simulări de rețele și sisteme cu milioane de entități (supercalculatoare)

Time Warp: ce se întâmplă la un mesaj „din trecut”



Stările vechi se păstrează până la **GVT** (global virtual time): momentul sub care niciun LP nu mai poate reveni – memoria de dinainte se poate elibera.

Pașii

1. LP 2 a avansat optimist până la 80.
2. Sosește un mesaj cu t = 60 („straggler”).
3. LP 2 revine la starea salvată de la 60 și trimite **anti-mesaje** pentru ce a trimis greșit.
4. Reia de la 60, de data aceasta corect.

Verificare

? În SimPy, ce face instrucțiunea yield env.timeout(5) într-un proces?

A oprește programul Python timp de 5 secunde reale

B suspendă procesul și îl reia după 5 unități de timp simulat

C creează 5 procese noi

D mută ceasul simulat înapoi cu 5 unități

Verificare

?

Semafor cu a = 30 s și b = 10 s. Pe roșu sosesc mașini la t = 10, 25 și 35. Când trece semaforul pe verde și cât durează verdele?

A la t = 10, timp de 10 s

B la t = 40, timp de 30 s

C la t = 35, timp de 30 s

D la t = 40, timp de 10 s

Verificare

? O linie are trei stații cu timpi medii 1,0, 1,1 și 0,9 minute. Care este throughput-ul **maxim** posibil, oricât de mari ar fi bufferele?

A 1,11 piese/min

B 1,00 piese/min

C ≈ 0,91 piese/min

D 0,33 piese/min

Rezumatul săptămânii 5

- ✓ **DES**: starea se schimbă doar la evenimente; ceasul sare între ele – exact și rapid
- ✓ Anatomie: ceas, **lista evenimentelor viitoare** (heap), acțiuni; condiție de oprire
- ✓ Statistici: medii **ponderate în timp** vs medii pe clienți; Little ca verificare
- ✓ Motor propriu în ~40 de linii: M/M/1, M/M/c, priorități, renunțări – validat pe formulele din S4
- ✓ Trei abordări: evenimente, activități, **procese**; SimPy = procese cu generatoare Python
- ✓ Aplicații: intersecție, linie de producție (buffere, gât de sticlă), urgențe (priorități), parcare
- ✓ La scară mare: unelte industriale, digital twins, simulare paralelă (conservativ vs Time Warp)

Săptămâna viitoare

Date de intrare, analiza rezultatelor, validare

- De unde vin distribuțiile: histograme, estimare, teste de potrivire
- Câte rulări? Încălzire, replicări, intervale de încredere
- Verificare și validare: când avem încredere într-un model?

Simulatorul nostru a dat W = 10,56 în loc de 10. Este o eroare sau doar zgomot statistic? Cum decidem?

Bibliografie pentru săptămâna 5

- Prof. Cernazean – Modelling and Simulation, Lecture 7: Discrete Events Simulation (materiale de curs)
- Ș. Holban – Tehnici de modelare și simulare, cap. 2: modelarea bazată pe evenimente, graful activităților
- R. J. Gibbens – Computer Systems Modelling, Cambridge – simularea cu evenimente discrete
- A. M. Law – Simulation Modeling and Analysis, ed. 5, McGraw-Hill, 2015
- J. Banks, J. S. Carson, B. L. Nelson, D. M. Nicol – Discrete-Event System Simulation, Pearson, 2010
- SimPy – documentația oficială, simpy.readthedocs.io
- L. Schruben – Simulation modeling with event graphs, Communications of the ACM, 1983
- R. M. Fujimoto – Parallel and Distributed Simulation Systems, Wiley, 2000
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5